

Week 05 — Statistical Control & Causality

Multiple regression, partialling out a control variable, and choosing controls with causal graphs

AUTHORS

Johann-Friedrich Salzmann

Joshua Elias Schmidt Rodrigues

DATE

2026-10-15

Topics:

Package management . . . 2
Where we left off . . . 2
The multiple regression model . . . 2
Data: German election results (2025) . . . 2
Turnout and AfD support . . . 4
Statistical control by hand . . . 8
Multiple regression with `lm()` . . . 15
Choosing control variables . . . 20
Data: polling stations on election day . . . 20
Causal graphs . . . 21
Two competing DAGs . . . 23
5 model variants . . . 26
Overview . . . 48
The true effect . . . 51
Concluding remarks . . . 54
Try it: AfD support, turnout and density . . . 54
Try it: the model variants under DAG A . . . 55
Try it: a care home in the district . . . 55

Package management

Next to `broom` and `modelsummary` from Week 4, we load `ggdag`, which draws causal graphs with ggplot2, and `dagitty`, which reads them.

```
if (!require(pacman)) install.packages("pacman")
```

```
MESSAGE
Loading required package: pacman
```

```
p_load(tidyverse, broom, modelsummary, ggdag, dagitty)
```

Where we left off

Week 4 fitted regression lines with a single predictor. One predictor mixes its own relationship with the outcome with everything else it goes along with. Today we add a second predictor to hold such a difference constant, and use causal graphs to decide which variables belong in the model.

The multiple regression model

With k predictors, the linear model becomes:

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + ... + \beta_k X_{ik} + \varepsilon_i$$

OLS works as before: it picks the $k + 1$ coefficients that minimize the sum of squared residuals.

$$\min_{\hat{\beta}_0,...,\hat{\beta}_k} \sum_{i=1}^{n} \left(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - ... - \hat{\beta}_k x_{ik} \right)^2$$

Each slope $\hat{\beta}_j$ is the average change in Y for a one-unit increase in X_j, **holding all other predictors constant**. That last part is what **statistical control** means.

Data: German election results (2025)

The GERDA counties from Week 4 again. As in Week 3, Berlin (state code `11`) is left out, since its East/West split doesn't map onto a single county. New is a **dummy variable** `east`: 1 for the counties of the former GDR, 0 for the West.

```
gerda <- read_csv("data/federal_cty_harm.csv")
```

MESSAGE

```
Rows: 4000 Columns: 127
-- Column specification --------------------------------------------------------
Delimiter: ","
chr    (2): county_code, state
dbl  (124): election_year, eligible_voters, number_voters, valid_votes, inva...
date   (1): election_date

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
east_codes <- c("12", "13", "14", "15", "16")

counties <- gerda |>
  filter(
    election_year == 2025,
    state != "11"
  ) |>
  mutate(
    density = population / area,
    east    = if_else(state %in% east_codes, 1, 0),
    region  = if_else(east == 1, "East", "West")
  ) |>
  select(
    county_code, state, east, region,
    cdu_csu, spd, greens = gruene, fdp, linke = linke_pds, afd, bsw,
    turnout, density
  ) |>
  mutate(across(cdu_csu:turnout, ~ .x * 100))
counties
```

OUTPUT

```
# A tibble: 399 x 13
   county_code state  east region cdu_csu   spd greens   fdp linke   afd   bsw
   <chr>       <chr> <dbl> <chr>    <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl>
 1 01001       01        0 West      17.4  15.7  20.6   3.30 12.9   13.8  3.33
 2 01002       01        0 West      18.7  19.3  23.1   3.83 14.4   10.9  3.36
 3 01003       01        0 West      21.9  21.4  18.3   3.79 10.9   15.3  3.78
 4 01004       01        0 West      25.3  19.6  10.5   4.14  8.06  22.3  4.62
 5 01051       01        0 West      30.3  17.7   9.15  5.51  6.10  21.1  3.51
 6 01053       01        0 West      28.8  18.5  13.2   5.07  7.12  19.1  3.80
 7 01054       01        0 West      29.1  17.3  13.4   4.80  6.17  14.5  3.04
 8 01055       01        0 West      31.2  19.9  12.6   5.16  5.92  17.6  3.09
 9 01056       01        0 West      28.9  20.2  14.5   5.06  7.57  15.6  3.57
10 01057       01        0 West      29.9  19.2  16.0   4.32  6.27  15.7  3.03
# i 389 more rows
# i 2 more variables: turnout <dbl>, density <dbl>
```

```
counties |>
  count(region)
```

```
# A tibble: 2 x 2
  region     n
  <chr>  <int>
1 East      75
2 West     324
```

One color per region, used in every plot below:

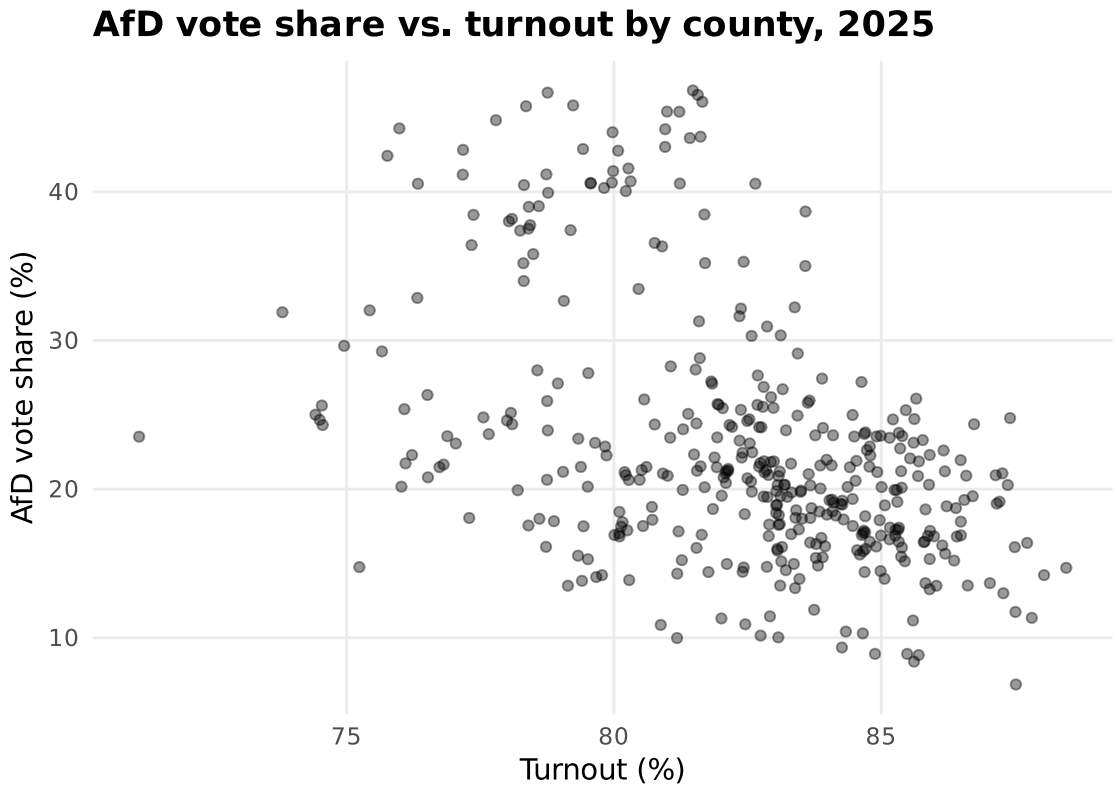
```
region_colors <- c(
  "West" = course_primary,
  "East" = "darkorange"
)
```

Turnout and AfD support

Does the AfD do better where fewer people vote? AfD share is our y, turnout our x:

$$\text{AfD}_i = \beta_0 + \beta_1 \text{Turnout}_i + \varepsilon_i$$

```
counties |>
  ggplot(aes(x = turnout, y = afd)) +
  geom_point(alpha = 0.4) +
  labs(
    title = "AfD vote share vs. turnout by county, 2025",
    x     = "Turnout (%)",
    y     = "AfD vote share (%)"
  ) +
  theme_intror()
```




```
bivariate_model <- lm(afd ~ turnout, data = counties)
bivariate_model |>
  tidy()
```

OUTPUT

```
# A tibble: 2 x 5
  term        estimate std.error statistic                              p.value
  <chr>          <dbl>     <dbl>     <dbl>                                <dbl>
1 (Intercept)   131.      10.4        12.7 0.00000000000000000000000000000319
2 turnout        -1.32     0.126     -10.5 0.000000000000000000000665
```

```r
bivariate_beta_0 <- bivariate_model |>
  tidy() |>
  filter(term == "(Intercept)") |>
  pull(estimate)
bivariate_beta_1 <- bivariate_model |>
  tidy() |>
  filter(term == "turnout") |>
  pull(estimate)
bivariate_beta_0
```

OUTPUT

```
[1] 131.4466
```

```r
bivariate_beta_1
```

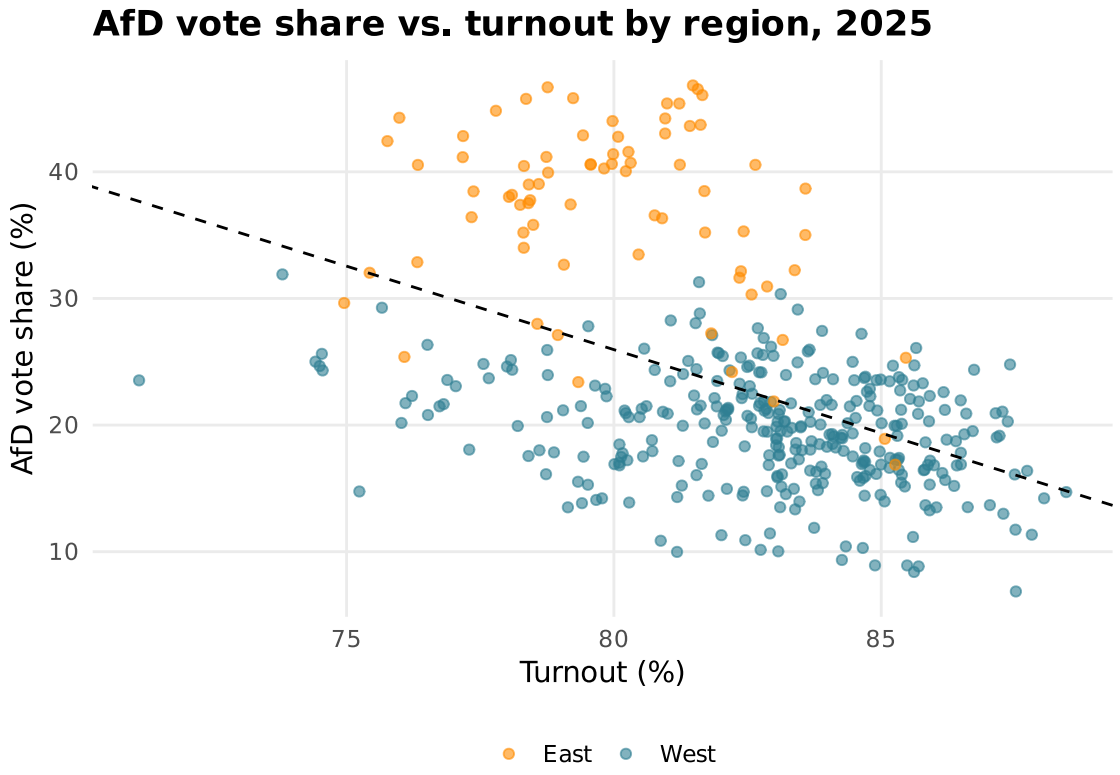
OUTPUT

```
[1] -1.318618
```

$$\widehat{\text{AfD}}_i = 131 - 1.32 \cdot \text{Turnout}_i$$

One percentage point more turnout goes along with 1.32 points less AfD support. Coloring the same points by region shows two separate clouds:

```r
counties |>
  ggplot(aes(x = turnout, y = afd, color = region)) +
  geom_point(alpha = 0.6) +
  geom_abline(
    intercept = bivariate_beta_0,
    slope     = bivariate_beta_1,
    linetype  = "dashed"
  ) +
  scale_color_manual(values = region_colors) +
  labs(
    title = "AfD vote share vs. turnout by region, 2025",
    x     = "Turnout (%)",
    y     = "AfD vote share (%)",
    color = NULL
  ) +
  theme_intror()
```



Eastern counties have lower turnout *and* much higher AfD shares. Within each region, the slope looks flatter than the dashed line through all counties. The region means:

$$\bar{y}_{\text{East}}, \quad \bar{y}_{\text{West}}$$

```r
region_means <- counties |>
  group_by(region) |>
  summarize(
    mean_afd     = mean(afd),
    mean_turnout = mean(turnout)
  )
region_means
```

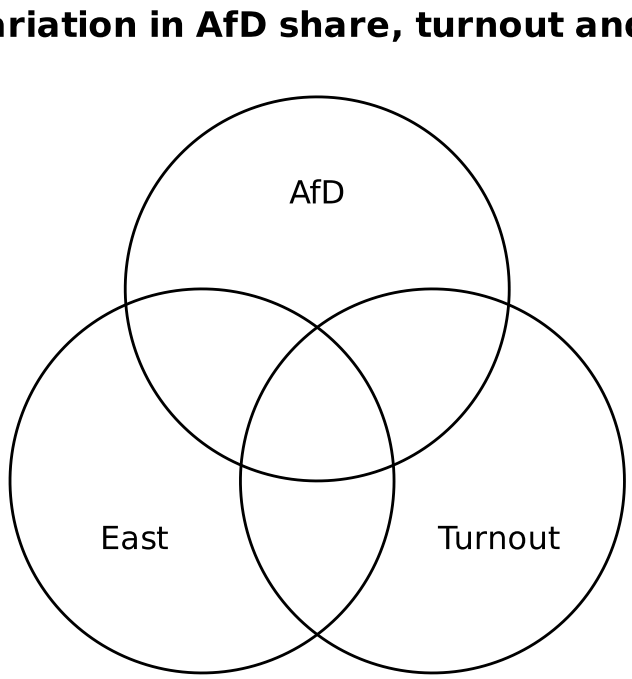
OUTPUT

```
# A tibble: 2 x 3
  region mean_afd mean_turnout
  <chr>     <dbl>        <dbl>
1 East       37.1         80.0
2 West       19.5         82.9
```

Statistical control by hand

How much of the turnout slope is left once we hold the region constant? Following the lecture, we purge `east` out of both variables and then regress what's left on each other, in three steps.

Venn diagrams help to follow along. Each circle stands for how much one variable differs from county to county. Where two circles overlap, knowing one variable helps to predict the other: the bigger the overlap, the better the prediction. Whatever lies outside all other circles, none of them can predict:



Step 1: AfD share purged of East

Regress AfD share on `east`:

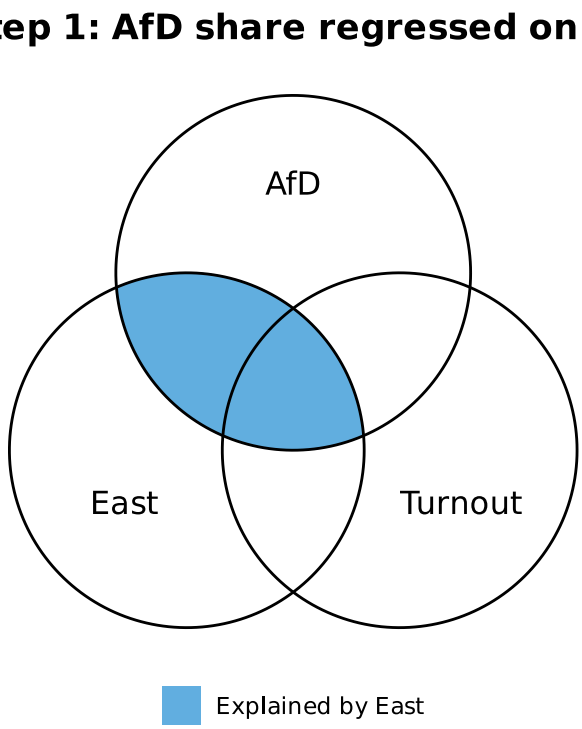
$$\text{AfD}_i = \gamma_0 + \gamma_1 \text{East}_i + u_i$$

```r
afd_east_model <- lm(afd ~ east, data = counties)
afd_east_model |>
  tidy()
```

OUTPUT

```
# A tibble: 2 x 5
  term        estimate std.error statistic                    p.value
  <chr>          <dbl>     <dbl>     <dbl>                      <dbl>
1 (Intercept)     19.5     0.277      70.5                   1.60e-226
2 east            17.5     0.640      27.4                   1.76e- 93
```

The blue overlap is the variation in AfD support that `east` explains:



With a dummy as the only predictor, the fitted values are just the two region means from above: $\hat{\gamma}_0$ is the West mean, and $\hat{\gamma}_1$ the East-West difference. The residuals are the part of AfD support that `east` doesn't explain: each county's distance from its own region's mean.

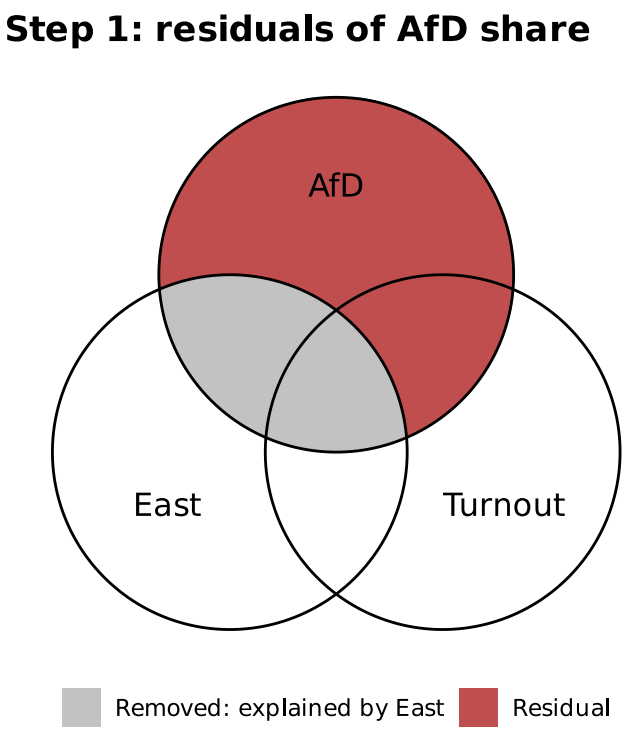
$$\widetilde{\text{AfD}}_i = \text{AfD}_i - \widehat{\text{AfD}}_i$$

```
partialled <- afd_east_model |>
  augment(data = counties) |>
  select(county_code, region, afd, turnout, afd_resid = .resid)
partialled
```

OUTPUT

```
# A tibble: 399 x 5
   county_code region   afd turnout afd_resid
   <chr>       <chr>  <dbl>   <dbl>     <dbl>
 1 01001       West    13.8    79.4    -5.71
 2 01002       West    10.9    82.5    -8.64
 3 01003       West    15.3    79.5    -4.26
 4 01004       West    22.3    76.2     2.75
 5 01051       West    21.1    82.1     1.53
 6 01053       West    19.1    84.1    -0.445
 7 01054       West    14.5    83.2    -5.00
 8 01055       West    17.6    82.9    -1.93
 9 01056       West    15.6    84.6    -3.94
10 01057       West    15.7    86.2    -3.89
# i 389 more rows
```

`afd_resid` is the red part: everything in the AfD circle outside the East circle. The grey part is what East already predicts, and it stays removed from here on. Part of the red still overlaps with turnout:



Step 2: turnout purged of East

The same for turnout:

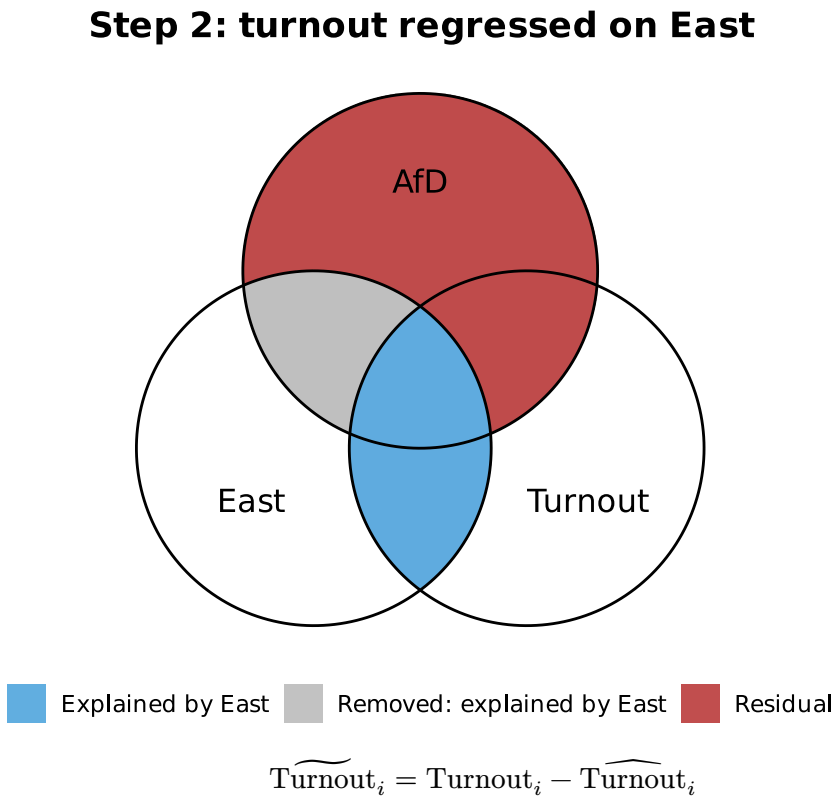
$$\text{Turnout}_i = \delta_0 + \delta_1 \text{East}_i + v_i$$

```r
turnout_east_model <- lm(turnout ~ east, data = counties)
turnout_east_model |>
  tidy()
```

OUTPUT

```
# A tibble: 2 x 5
  term        estimate std.error statistic             p.value
  <chr>          <dbl>     <dbl>     <dbl>               <dbl>
1 (Intercept)    82.9      0.154    539.   0
2 east           -2.95     0.355     -8.31 0.00000000000000158
```

This time the blue overlap is the variation in turnout that `east` explains:

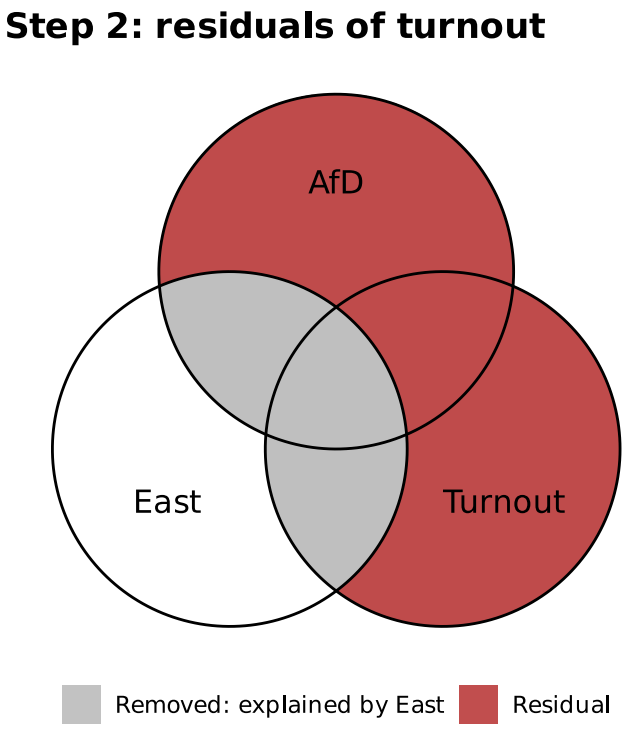


```r
partialled <- turnout_east_model |>
  augment(data = partialled) |>
  select(county_code, region, afd, turnout, afd_resid, turnout_resid = .resid)
partialled
```

OUTPUT

```
# A tibble: 399 x 6
   county_code region   afd turnout afd_resid turnout_resid
   <chr>       <chr>  <dbl>   <dbl>     <dbl>         <dbl>
 1 01001       West    13.8    79.4    -5.71        -3.52
 2 01002       West    10.9    82.5    -8.64        -0.461
 3 01003       West    15.3    79.5    -4.26        -3.40
 4 01004       West    22.3    76.2     2.75        -6.69
 5 01051       West    21.1    82.1     1.53        -0.829
 6 01053       West    19.1    84.1    -0.445        1.19
 7 01054       West    14.5    83.2    -5.00         0.298
 8 01055       West    17.6    82.9    -1.93        -0.0130
 9 01056       West    15.6    84.6    -3.94         1.68
10 01057       West    15.7    86.2    -3.89         3.28
# i 389 more rows
```

`turnout_resid` is the turnout circle without the East circle. Both residuals together:



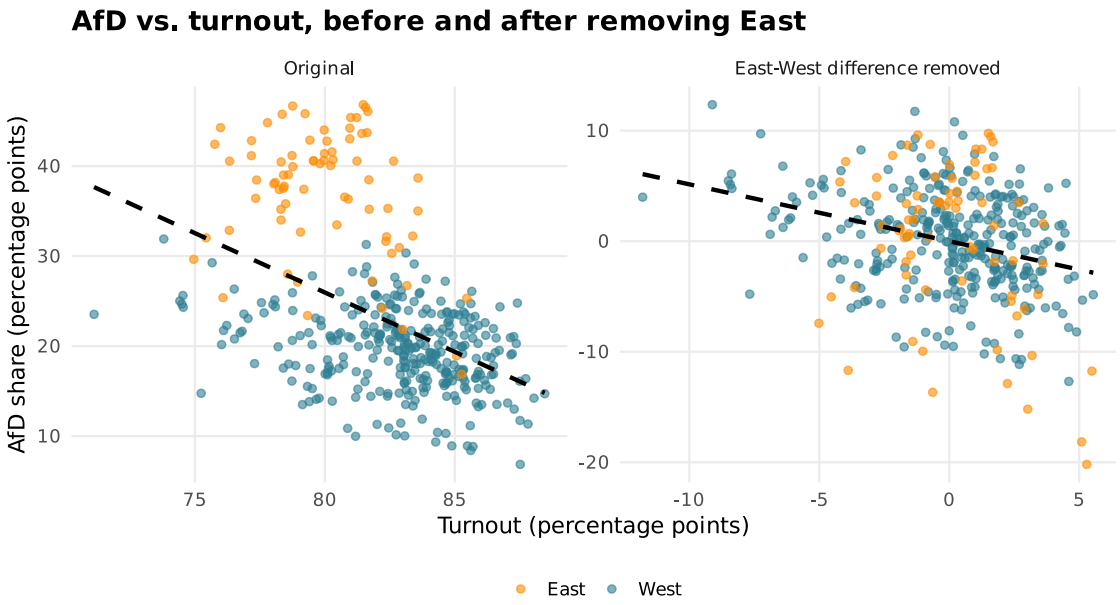
Step 3: regressing the residuals on each other

Both residual columns are now free of the East-West difference. With the region means subtracted, the two clouds sit on top of each other. Next to the original data from above, each with a dashed OLS fit through all counties:

```r
original_panel <- partialled |>
  mutate(
    panel = "Original",
    x     = turnout,
    y     = afd
  )
residual_panel <- partialled |>
  mutate(
    panel = "East-West difference removed",
    x     = turnout_resid,
    y     = afd_resid
  )

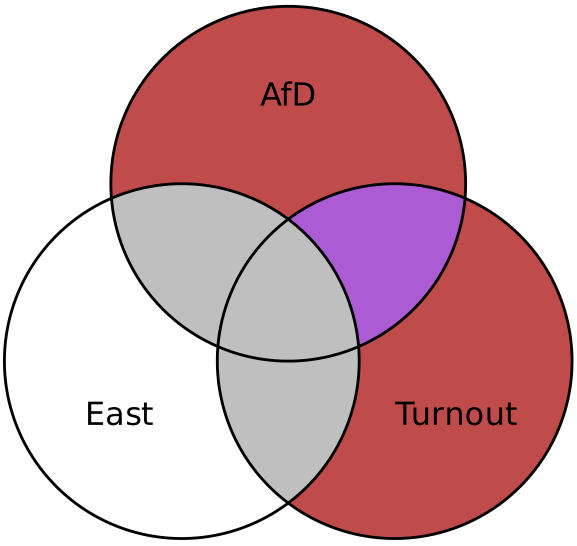
partialled_panels <- bind_rows(original_panel, residual_panel) |>
  mutate(panel = factor(panel, levels = c("Original", "East-West difference removed")))
```

```
partialled_panels |>
  ggplot(aes(x = x, y = y)) +
  geom_point(aes(color = region), alpha = 0.6) +
  geom_smooth(
    method   = "lm",
    formula  = y ~ x,
    se       = FALSE,
    color    = "black",
    linetype = "dashed"
  ) +
  facet_wrap(~ panel, scales = "free") +
  scale_color_manual(values = region_colors) +
  labs(
    title = "AfD vs. turnout, before and after removing East",
    x     = "Turnout (percentage points)",
    y     = "AfD share (percentage points)",
    color = NULL
  ) +
  theme_intror()
```



Regressing one residual on the other uses only the purple part: the variation AfD share and turnout have in common *outside* the East circle. That is the turnout slope with `east` held constant:

Step 3: residuals regressed on each other



Removed: explained by East | Residual | Turnout slope

$$\widetilde{\text{AfD}}_i = \alpha_0 + \beta_1 \widetilde{\text{Turnout}}_i + w_i$$

```
partialled_model <- lm(afd_resid ~ turnout_resid, data = partialled)
partialled_model |>
  tidy()
```

OUTPUT

```
# A tibble: 2 x 5
  term                     estimate std.error           statistic       p.value
  <chr>                       <dbl>     <dbl>               <dbl>         <dbl>
1 (Intercept)   -0.0000000000000121    0.240  -0.0000000000000506 1.000
2 turnout_resid -0.514                 0.0868 -5.93                0.00000000660
```

```
partialled_beta_1 <- partialled_model |>
  tidy() |>
  filter(term == "turnout_resid") |>
  pull(estimate)
partialled_beta_1
```

OUTPUT

```
[1] -0.5144773
```

The slope dropped from -1.32 to -0.51. Most of the bivariate relationship was the East-West difference in both variables.

Multiple regression with `lm()`

`lm()` does all three steps at once: add the control with `+` in the formula.

$$\text{AfD}_i = \beta_0 + \beta_1 \text{Turnout}_i + \beta_2 \text{East}_i + \varepsilon_i$$

```
control_model <- lm(afd ~ turnout + east, data = counties)
control_model |>
  tidy()
```

OUTPUT

```
# A tibble: 3 x 5
  term        estimate std.error statistic                              p.value
  <chr>          <dbl>     <dbl>     <dbl>                                <dbl>
1 (Intercept)   62.2      7.21        8.63                              1.51e-16
2 turnout       -0.514    0.0869     -5.92                              6.90e- 9
3 east          16.0      0.665      24.1                               1.80e-79
```

```
control_beta_0 <- control_model |>
  tidy() |>
  filter(term == "(Intercept)") |>
  pull(estimate)
control_beta_1 <- control_model |>
  tidy() |>
  filter(term == "turnout") |>
  pull(estimate)
control_beta_2 <- control_model |>
  tidy() |>
  filter(term == "east") |>
  pull(estimate)
control_beta_0
```

OUTPUT

```
[1] 62.20589
```

```
control_beta_1
```

OUTPUT

```
[1] -0.5144773
```

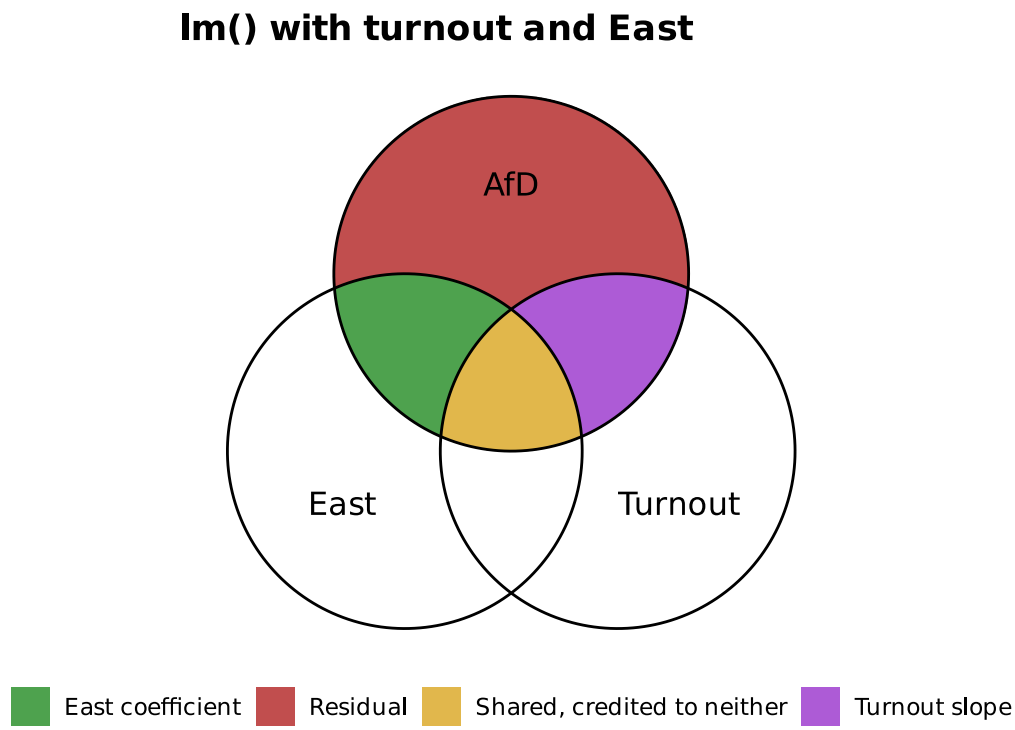
```
control_beta_2
```

OUTPUT

```
[1] 16.00378
```

$$\widehat{\text{AfD}}_i = 62.2 - 0.514 \cdot \text{Turnout}_i + 16 \cdot \text{East}_i$$

The turnout slope is exactly the one from the three steps. In the Venn diagram, `lm()` credits each coefficient only with what that predictor predicts on its own. The part of AfD support that turnout and East predict together goes to neither coefficient:



Both ways give the same turnout slope:

```
tibble(
  three_steps = partialled_beta_1,
  lm          = control_beta_1
)
```

OUTPUT

```
# A tibble: 1 x 2
  three_steps     lm
        <dbl>  <dbl>
1      -0.514 -0.514
```

Interpreting a dummy coefficient

A dummy is an indicator: it is 1 if a condition holds and 0 otherwise. Written with the indicator function $\mathbb{1}(\cdot)$, the fitted model reads:

$$\widehat{\text{AfD}}_i = \hat{\beta}_0 + \hat{\beta}_1 \text{Turnout}_i + \hat{\beta}_2 \cdot \mathbb{1}(\text{county } i \text{ is in the East})$$

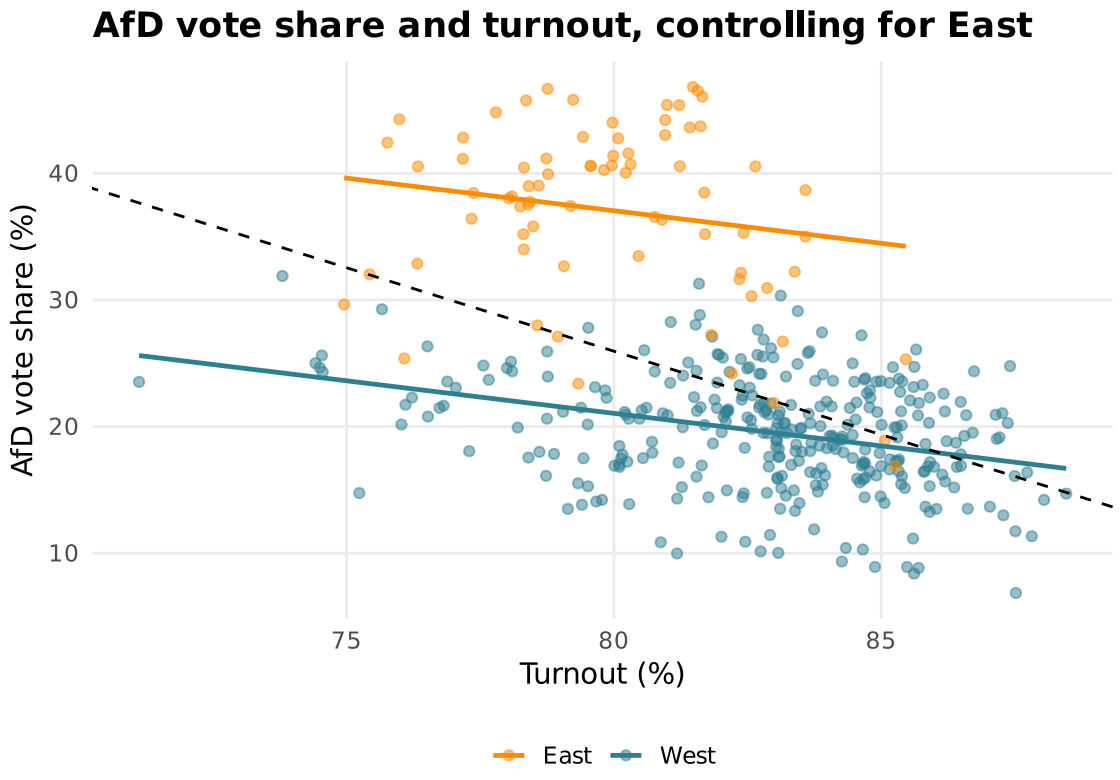
So a dummy shifts the intercept. Plugging in both values gives one line per region, with the same slope:

$$\widehat{\text{AfD}}_i = \begin{cases} \hat{\beta}_0 + \hat{\beta}_1 \text{Turnout}_i & \text{if West} \\ \left(\hat{\beta}_0 + \hat{\beta}_2\right) + \hat{\beta}_1 \text{Turnout}_i & \text{if East} \end{cases}$$

At the same turnout, an Eastern county is predicted to vote 16 points more AfD. `augment()` draws both lines, next to the dashed bivariate one:

```r
control_fit <- control_model |>
  augment(data = counties)
```

```r
control_fit |>
  ggplot(aes(x = turnout, color = region)) +
  geom_point(aes(y = afd), alpha = 0.5) +
  geom_line(aes(y = .fitted), linewidth = 1) +
  geom_abline(
    intercept = bivariate_beta_0,
    slope     = bivariate_beta_1,
    linetype  = "dashed"
  ) +
  scale_color_manual(values = region_colors) +
  labs(
    title = "AfD vote share and turnout, controlling for East",
    x     = "Turnout (%)",
    y     = "AfD vote share (%)",
    color = NULL
  ) +
  theme_intror()
```



Comparing the models

Week 4 left adjusted R^2 for models with several predictors. It penalizes each additional predictor, so it only rises when a predictor explains enough to be worth it:

```r
bivariate_glance <- bivariate_model |>
  glance() |>
  mutate(model = "Turnout only")
control_glance <- control_model |>
  glance() |>
  mutate(model = "Turnout + East")

bind_rows(bivariate_glance, control_glance) |>
  select(model, r.squared, adj.r.squared)
```

OUTPUT

```
# A tibble: 2 x 3
  model          r.squared adj.r.squared
  <chr>              <dbl>         <dbl>
1 Turnout only       0.217         0.215
2 Turnout + East     0.682         0.680
```

Side by side as a regression table:

```r
turnout_models <- list(
  "Turnout only"   = bivariate_model,
  "Turnout + East" = control_model
)

modelsummary(
  turnout_models,
  output    = "tinytable",
  coef_map  = c(
    "(Intercept)" = "Intercept",
    "turnout"     = "Turnout (%)",
    "east"        = "East"
  ),
  statistic = NULL,
  stars     = FALSE,
  gof_map   = c("nobs", "r.squared", "adj.r.squared"),
  fmt       = 3
)
```

	Turnout only	Turnout + East
Intercept	131.447	62.206
Turnout (%)	-1.319	-0.514
East		16.004
Num.Obs.	399	399
R2	0.217	0.682
R2 Adj.	0.215	0.680

Choosing control variables

Above, we held `east` constant as if it obviously belonged in the model. Whether a variable belongs in a model is a causal question, and the data alone can't answer it. We switch to a small, well-defined example to see why.

Data: polling stations on election day

A large German city's election office ran 300 polling districts (*Wahlbezirke*, the smallest electoral units) in the last federal election. Do more voting booths shorten the queue? Per district, the office recorded:

Column	Meaning
`district`	District number
`eligible`	Eligible voters
`last_turnout`	Turnout at the previous federal election (%)
`booths`	Voting booths set up
`wait`	Average waiting time at the midday peak (minutes)
`observer`	1 if an observer from the election office visited, 0 otherwise

```
stations <- read_csv("w05/polling_stations.csv")
```

MESSAGE

```
Rows: 300 Columns: 6
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (6): district, eligible, last_turnout, booths, wait, observer

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
stations
```

```
# A tibble: 300 x 6
   district eligible last_turnout booths  wait observer
      <dbl>    <dbl>        <dbl>  <dbl> <dbl>    <dbl>
 1        1     1760         74.5      4    16        1
 2        2     1653         67.4      3    12        0
 3        3     1995         84.4      5    16        1
 4        4     1293         60.6      1    15        1
 5        5     1847         72.6      4    18        1
 6        6     1173         78.4      2     6        0
 7        7     1457         65.4      2     8        0
 8        8     2120         70.3      4    21        1
 9        9     1908         68        3    17        1
10       10     2211         70.6      4    16        1
# i 290 more rows
```

```
stations |>
  count(booths)
```

```
# A tibble: 6 x 2
  booths     n
   <dbl> <int>
1      1    18
2      2    72
3      3    94
4      4    86
5      5    26
6      6     4
```

What we know about how the office works: it allocates booths by the number of eligible voters. Shortly after noon, it sends an observer to every station that reports a queue of more than 12 minutes, and to every large station, meaning one with 6 booths. Observers only watch and take notes.

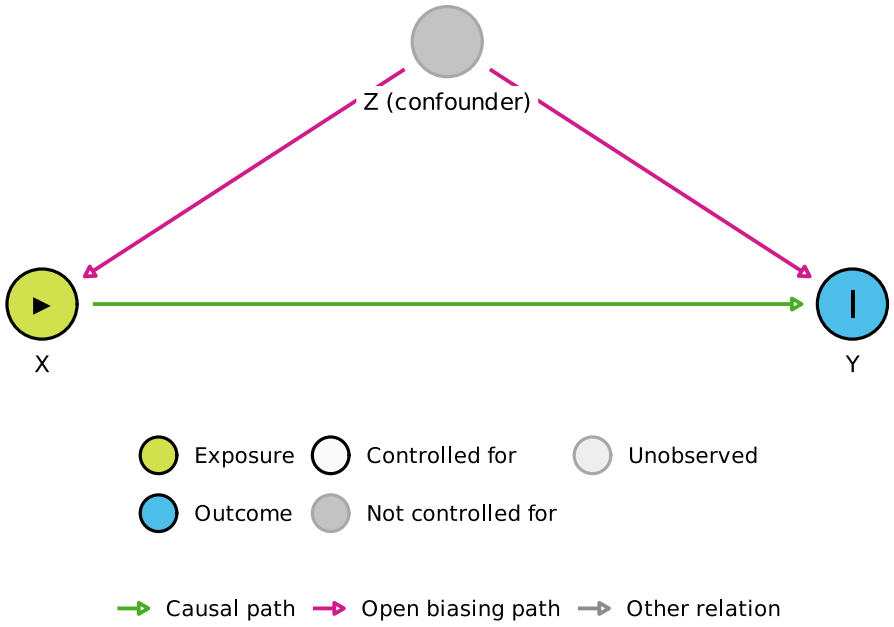
Causal graphs

A **directed acyclic graph (DAG)** draws our assumptions about what causes what: each node is a variable, each arrow a direct causal effect, and no chain of arrows leads back to where it started. Any sequence of arrows connecting X and Y, whichever way they point, is a **path**. A path that starts with an arrow *into* X is a **backdoor path**: it creates an association between X and Y that isn't X's effect.

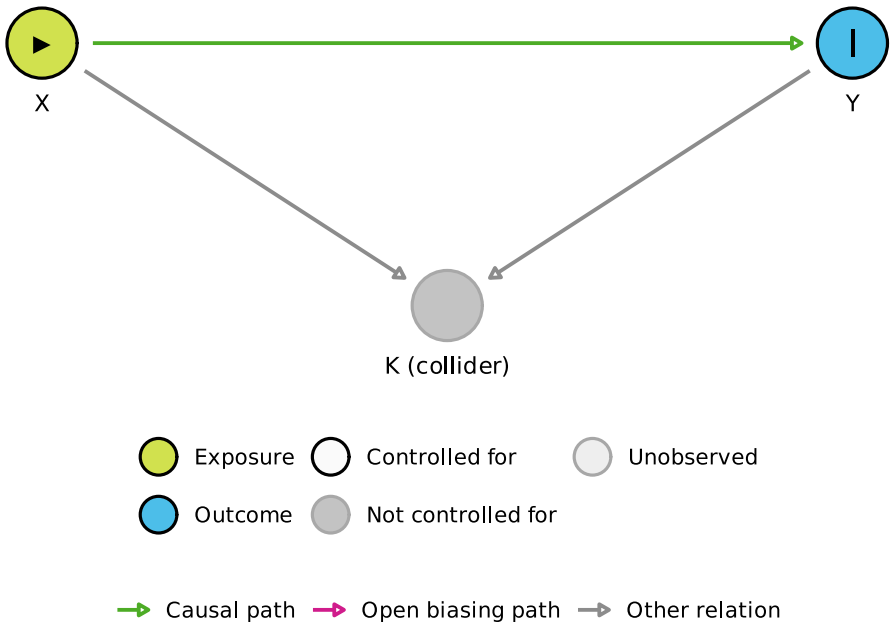
Two building blocks decide whether a path is open:

- A **confounder** Z sits on a fork, $X \leftarrow Z \rightarrow Y$. The path is open, and controlling for Z closes it.
- A **collider** K sits where two arrows meet, $X \rightarrow K \leftarrow Y$. The path is closed, and controlling for K opens it.

With nothing controlled for, the confounder's backdoor path is open (pink) and biases the estimate of X's effect on Y:



The collider's path stays closed (grey) as long as we leave K out of the model:



To estimate the effect of X on Y, we need a set of controls that closes every backdoor path without opening a new one, an **adjustment set**.

Drawing DAGs with dagitty

`dagitty` reads a DAG written in its own model code: one line per node (with optional attributes in brackets), one line per arrow. Each DAG's model code below also works in the browser: paste the text between `dagitty('` and `')` into the "Model code" box at dagitty.net to move nodes around and click through controls interactively.

To draw the same DAGs here, `include/plot_dag.R` provides `plot_dag()`, which uses dagitty.net's styles: green with a triangle for the exposure, blue with a bar for the outcome, white for what we control for, and light grey for what we don't observe. Arrows on the causal path are green, and arrows on an open path that biases the estimate are pink. It takes the DAG (what we control for is marked `[adjusted]` in its model code) and, optionally, readable node labels; without them, it shows the variable names:

```
source("include/plot_dag.R")

node_labels <- tribble(
  ~name,           ~label,
  "booths",        "Booths",
  "wait",          "Waiting time",
  "eligible",      "Eligible voters",
  "observer",      "Observer visit",
  "usual_turnout", "Usual turnout",
  "last_turnout",  "Last turnout"
)
```

Two competing DAGs

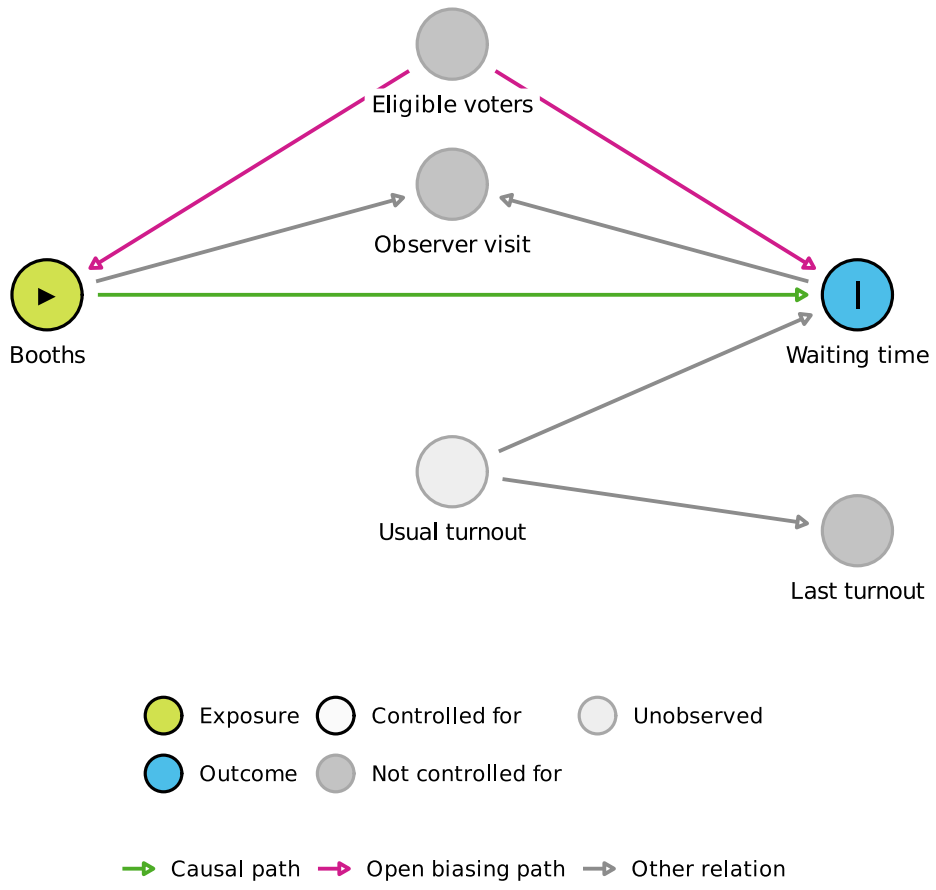
Usual turnout is how many of a district's eligible voters habitually go to vote. The office knows it from experience, but it is not recorded anywhere. Most arrows are uncontroversial:

- Eligible → Booths: the office allocates booths by the number of eligible voters.
- Eligible → Wait: more eligible voters means more people in the queue.
- Booths → Wait: each booth processes voters in parallel, so more booths shorten the queue. This is the effect we want.
- UsualTurnout → Wait: where people usually vote a lot, more of the eligible voters show up today, and the queue gets longer.
- UsualTurnout → LastTurnout: the same voting habit produced last election's turnout.
- Booths → Observer: every large station gets an observer.
- Wait → Observer: every station reporting a long queue gets an observer.

The disputed arrow is UsualTurnout → Booths. In **DAG A**, the office follows a strict booths-per-voter rule, and the arrow doesn't exist:

```
dag_a <- dagitty('dag {
  booths        [exposure, pos="0,0"]
  wait          [outcome, pos="3,0"]
  eligible      [pos="1.5,-1.7"]
  observer      [pos="1.5,-0.75"]
  usual_turnout [latent, pos="1.5,1.2"]
  last_turnout  [pos="3,1.6"]
  eligible      -> booths
  eligible      -> wait
  booths        -> wait
  booths        -> observer
  wait          -> observer
  usual_turnout -> wait
  usual_turnout -> last_turnout
}')
```

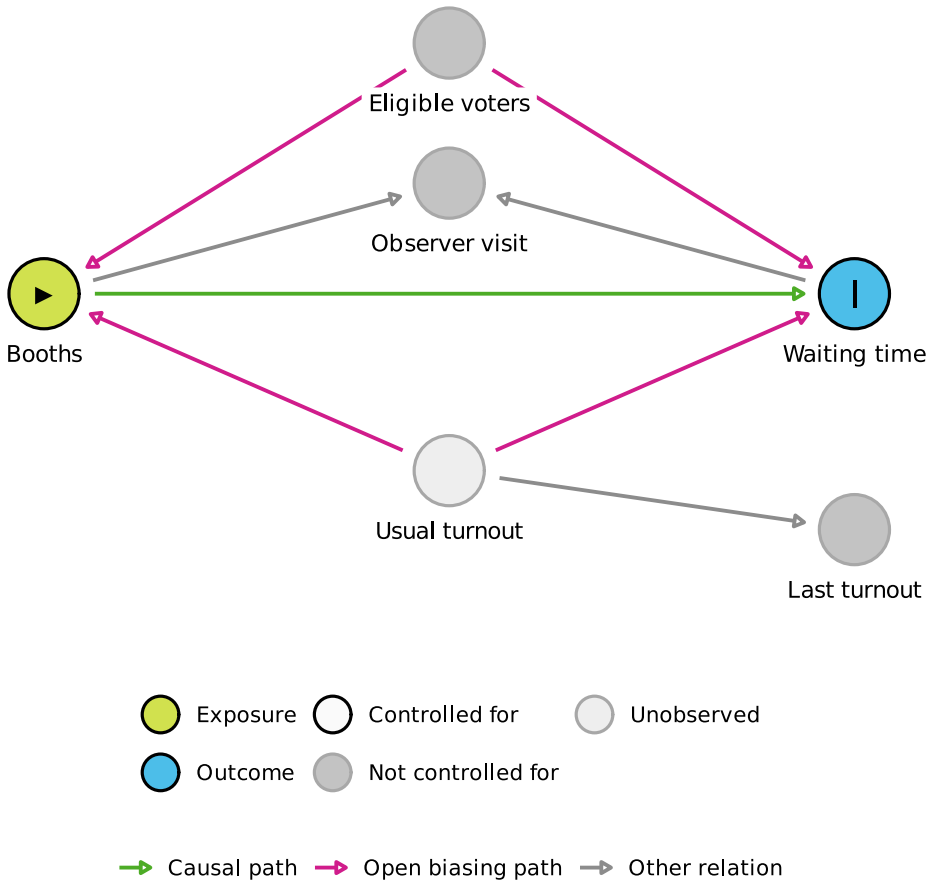
```
plot_dag(dag_a, labels = node_labels)
```



In **DAG B**, officials know their districts and add booths where turnout is usually high:

```
dag_b <- dagitty('dag {
  booths        [exposure, pos="0,0"]
  wait          [outcome, pos="3,0"]
  eligible      [pos="1.5,-1.7"]
  observer      [pos="1.5,-0.75"]
  usual_turnout [latent, pos="1.5,1.2"]
  last_turnout  [pos="3,1.6"]
  eligible      -> booths
  eligible      -> wait
  booths        -> wait
  booths        -> observer
  wait          -> observer
  usual_turnout -> wait
  usual_turnout -> last_turnout
  usual_turnout -> booths
}')
```

```
plot_dag(dag_b, labels = node_labels)
```



`adjustmentSets()` lists the minimal adjustment sets among the observed variables. Under DAG A, eligible voters are enough:

```
adjustmentSets(dag_a)
```

```
>_ OUTPUT
{ eligible }
```

Under DAG B, there is none: the backdoor path through usual turnout can only be closed by controlling for usual turnout itself, which we don't observe:

```
dag_b |>
  adjustmentSets() |>
  length()
```

```
>_ OUTPUT
[1] 0
```

Nothing in the data tells us which DAG is right. It's a claim about how the election office works, and choosing one is an assumption we have to state and defend. We go with **DAG B** from here on.

5 model variants

We estimate the effect of booths on waiting time five times, each with a different set of controls. Every variant follows the same pattern: the model, its DAG with the controls marked, `dagitty`'s verdict, the regression, a plot, and the booth coefficient next to the true effect.

Variant	Controls	Control types	`dagitty` verdict
1	none		two backdoor paths open
2	Elig	Elig: confounder	one backdoor path still open
3	Elig, T-Usual*	Elig: confounder; T-Usual: unobserved confounder, imagined observed	valid adjustment set
4	Elig, T-Last	Elig: confounder; T-Last: proxy for usual turnout	one backdoor path still open
5	Elig, T-Last, Obs	Elig: confounder; T-Last: proxy for usual turnout; Obs: collider	collider path opened

Elig = eligible voters, T-Usual = usual turnout, T-Last = last turnout, Obs = observer visit. Model names in tables and plots list their controls the same way. * Usual turnout is unobserved in practice: variant 3 is impossible with real data, a thought experiment we can only run because of how the data came about.

In every DAG below, the controls are marked `[adjusted]` in the model code, so pasting it into dagitty.net reproduces the plot.

To draw a model with several predictors as a line, we predict the waiting time for 1 to 6 booths, holding every control at its average:

```
prediction_grid <- stations |>
  summarize(
    eligible     = mean(eligible),
    last_turnout = mean(last_turnout)
  ) |>
  expand_grid(booths = 1:6)
prediction_grid
```

OUTPUT

```
# A tibble: 6 x 3
  eligible last_turnout booths
     <dbl>        <dbl>  <int>
1    1615.         71.7      1
2    1615.         71.7      2
3    1615.         71.7      3
4    1615.         71.7      4
5    1615.         71.7      5
6    1615.         71.7      6
```

One color per model variant, used in every plot below:

```
variant_colors <- c(
  "1: none"                          = "grey50",
  "2: Elig"                          = "#3a9ad9",
  "3: Elig, T-Usual*"                = "forestgreen",
  "4: Elig, T-Last"                  = "darkorchid",
  "5: Elig, T-Last, Obs (no visit)" = "goldenrod",
  "5: Elig, T-Last, Obs (visit)"    = "darkorange"
)
```

Model variant 1: no controls

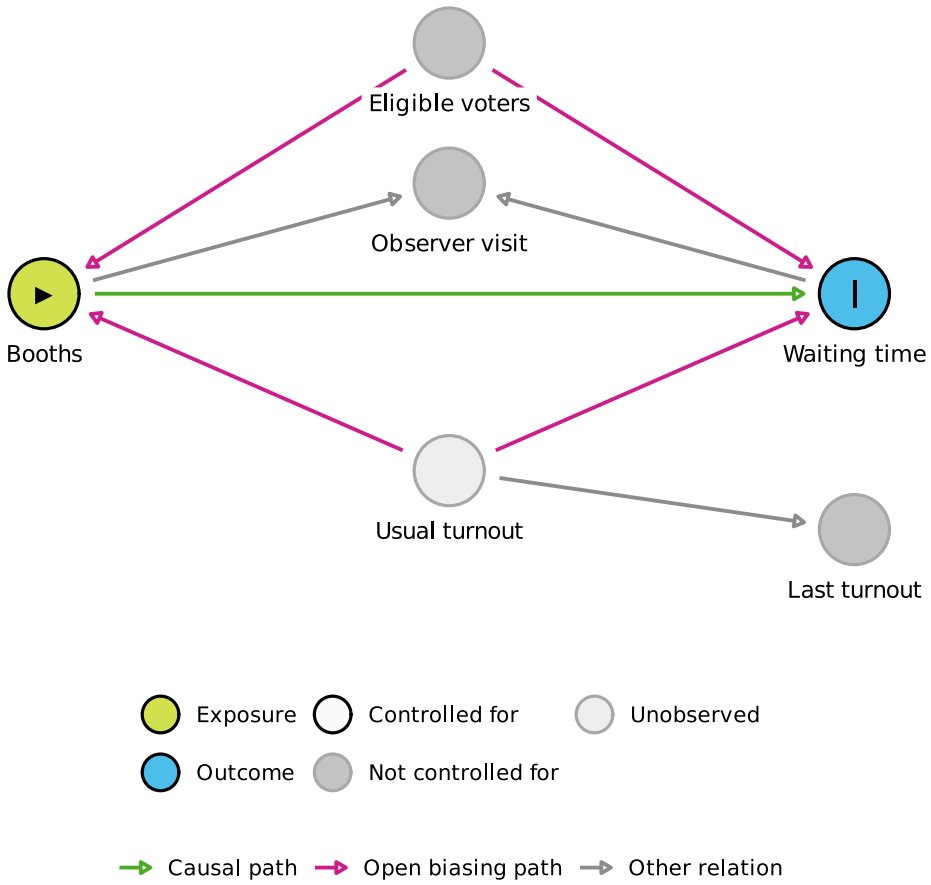
The simplest model regresses waiting time on booths alone, without any control variable:

$$\text{Wait}_i = \beta_0 + \beta_1 \text{Booths}_i + \varepsilon_i$$

Nothing is controlled for in the DAG:

```
dag_v1 <- dagitty('dag {
  booths        [exposure, pos="0,0"]
  wait          [outcome, pos="3,0"]
  eligible      [pos="1.5,-1.7"]
  observer      [pos="1.5,-0.75"]
  usual_turnout [latent, pos="1.5,1.2"]
  last_turnout  [pos="3,1.6"]
  eligible      -> booths
  eligible      -> wait
  booths        -> wait
  booths        -> observer
  wait          -> observer
  usual_turnout -> wait
  usual_turnout -> last_turnout
  usual_turnout -> booths
}')
```

```
plot_dag(dag_v1, labels = node_labels)
```



`paths()` lists every path between booths and waiting time and whether it is open, given the controls. Both backdoor paths are open; only the path through the observer visit is closed, since it runs through a collider:

```
paths(dag_v1, Z = adjustedNodes(dag_v1)) |>
  as_tibble()
```

OUTPUT

```
# A tibble: 4 x 2
  paths                            open
  <chr>                            <lgl>
1 booths -> observer <- wait       FALSE
2 booths -> wait                   TRUE
3 booths <- eligible -> wait       TRUE
4 booths <- usual_turnout -> wait TRUE
```

With nothing controlled for, there is no adjustment set:

```
isAdjustmentSet(dag_v1, adjustedNodes(dag_v1))
```

OUTPUT

```
[1] FALSE
```

```
model_v1 <- lm(wait ~ booths, data = stations)
model_v1 |>
  tidy()
```

OUTPUT

```
# A tibble: 2 x 5
  term        estimate std.error statistic              p.value
  <chr>          <dbl>     <dbl>     <dbl>                <dbl>
1 (Intercept)     7.96     0.914      8.71 0.000000000000000211
2 booths          1.33     0.275      4.83 0.00000217
```

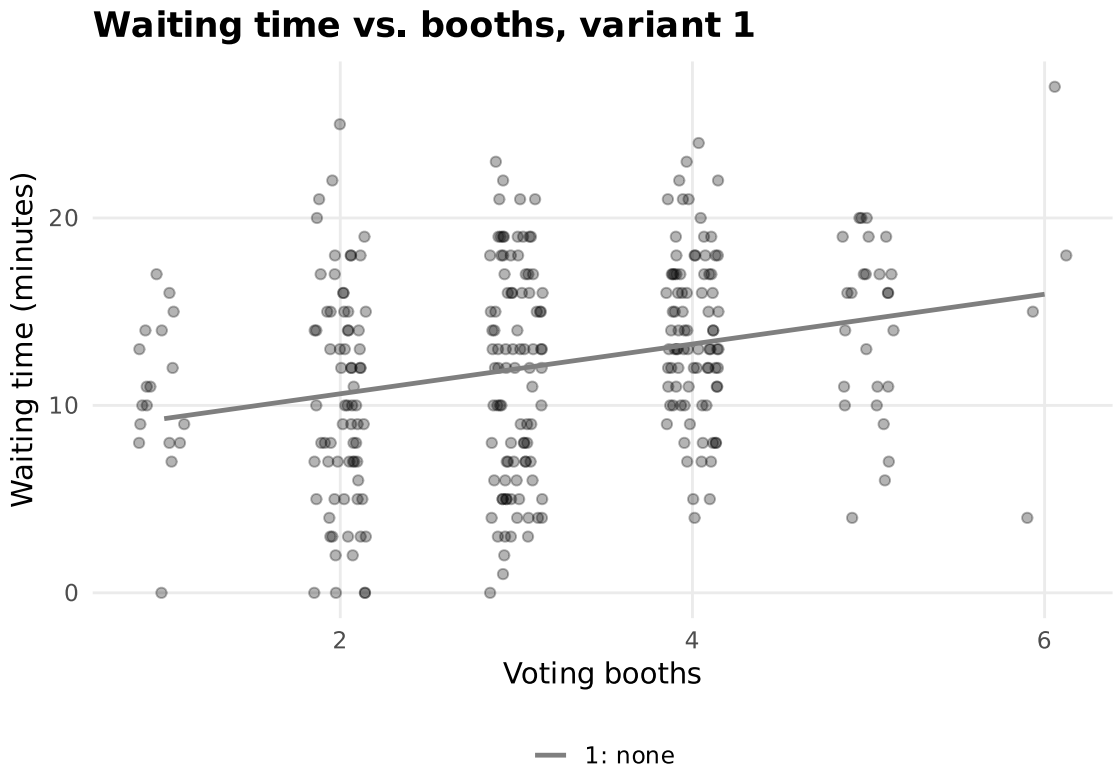
```
beta_v1 <- model_v1 |>
  tidy() |>
  filter(term == "booths") |>
  pull(estimate)
beta_v1
```

OUTPUT

```
[1] 1.327584
```

```
lines_v1 <- model_v1 |>
  augment(newdata = prediction_grid) |>
  mutate(model = "1: none")
```

```
stations |>
  ggplot(aes(x = booths)) +
  geom_jitter(aes(y = wait), width = 0.15, height = 0, alpha = 0.3) +
  geom_line(aes(y = .fitted, color = model), data = lines_v1, linewidth = 1) +
  scale_color_manual(values = variant_colors) +
  labs(
    title = "Waiting time vs. booths, variant 1",
    x     = "Voting booths",
    y     = "Waiting time (minutes)",
    color = NULL
  ) +
  theme_intror()
```



A running plot collects the booth coefficient from every variant. With `draw = FALSE`, `modelplot()` returns its estimates as a tibble instead of a plot, so we can draw them ourselves. The bars are 95% confidence intervals (Week 3); how they are computed for regression coefficients comes in Week 6. The red line marks the true effect of a booth; how we can know it, we reveal at the end:

```r
variant_models <- list("1: none" = model_v1)

modelplot(variant_models, coef_map = c(booths = "Booths"), draw = FALSE) |>
  ggplot(aes(x = estimate, y = fct_rev(model))) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  geom_vline(xintercept = -3, color = "firebrick", linewidth = 1) +
    geom_errorbar(aes(xmin = conf.low, xmax = conf.high), width = 0.2, color =
course_primary) +
  geom_point(size = 3, color = course_primary) +
  labs(
    title = "Estimated vs. true effect of one booth",
    x     = "Minutes of waiting time",
    y     = NULL
  ) +
  theme_intror()
```



More booths, *longer* queues: one more booth goes along with 1.33 more minutes of waiting. Big districts and districts that usually vote a lot get more booths *and* have longer queues, and both open backdoor paths flip the sign.

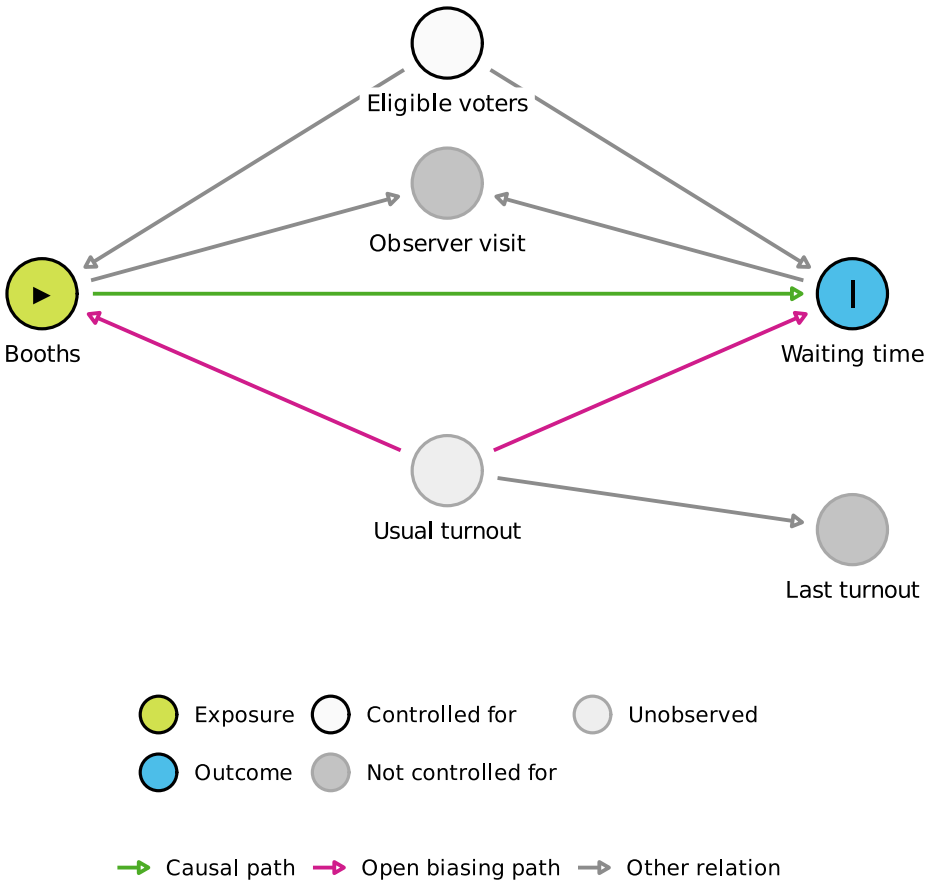
Model variant 2: controlling for eligible voters

Eligible voters are a **confounder**: they cause both the number of booths and the waiting time. Controlling for them closes that backdoor path:

$$\text{Wait}_i = \beta_0 + \beta_1 \text{Booths}_i + \beta_2 \text{Eligible}_i + \varepsilon_i$$

```
dag_v2 <- dagitty('dag {
  booths        [exposure, pos="0,0"]
  wait          [outcome, pos="3,0"]
  eligible      [adjusted, pos="1.5,-1.7"]
  observer      [pos="1.5,-0.75"]
  usual_turnout [latent, pos="1.5,1.2"]
  last_turnout  [pos="3,1.6"]
  eligible      -> booths
  eligible      -> wait
  booths        -> wait
  booths        -> observer
  wait          -> observer
  usual_turnout -> wait
  usual_turnout -> last_turnout
  usual_turnout -> booths
}')
```

```
plot_dag(dag_v2, labels = node_labels)
```



```
paths(dag_v2, Z = adjustedNodes(dag_v2)) |>
  as_tibble()
```

OUTPUT

```
# A tibble: 4 x 2
  paths                            open
  <chr>                            <lgl>
1 booths -> observer <- wait       FALSE
2 booths -> wait                   TRUE
3 booths <- eligible -> wait       FALSE
4 booths <- usual_turnout -> wait TRUE
```

The path through usual turnout stays open, so eligible voters alone are not an adjustment set:

```
isAdjustmentSet(dag_v2, adjustedNodes(dag_v2))
```

OUTPUT

```
[1] FALSE
```

```
model_v2 <- lm(wait ~ booths + eligible, data = stations)
model_v2 |>
  tidy()
```

OUTPUT

```
# A tibble: 3 x 5
  term        estimate std.error statistic                                p.value
  <chr>          <dbl>     <dbl>     <dbl>                                  <dbl>
1 (Intercept)  -3.59    0.989        -3.63                                3.34e- 4
2 booths       -1.42    0.265        -5.33                                1.92e- 7
3 eligible      0.0125  0.000784     15.9                                 9.25e-42
```

```
beta_v2 <- model_v2 |>
  tidy() |>
  filter(term == "booths") |>
  pull(estimate)
beta_v2
```

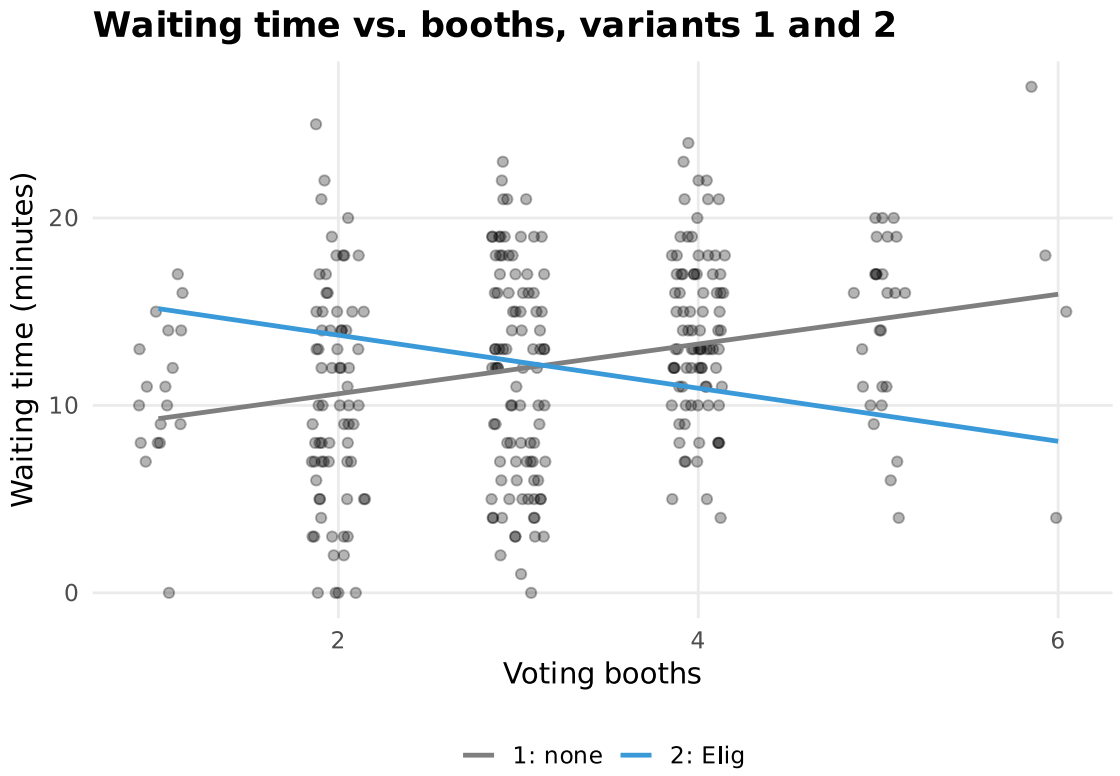
OUTPUT

```
[1] -1.415122
```

```
lines_v2 <- model_v2 |>
  augment(newdata = prediction_grid) |>
  mutate(model = "2: Elig")

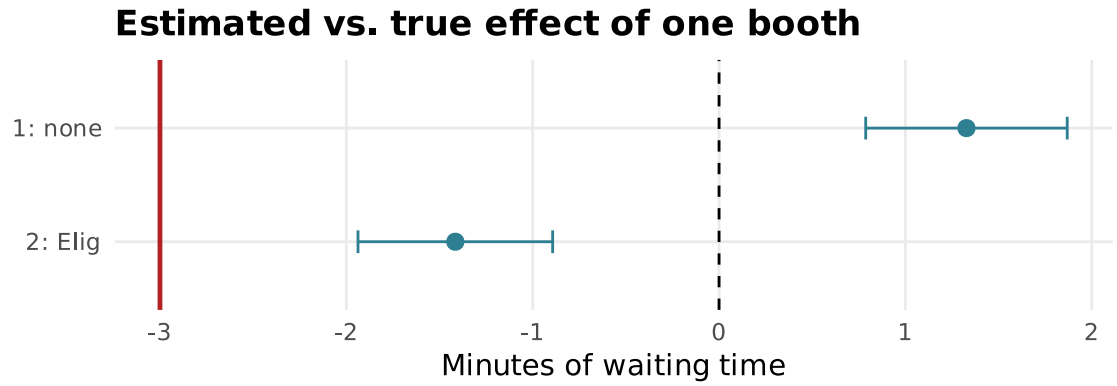
lines_v1_v2 <- bind_rows(lines_v1, lines_v2)
```

```
stations |>
  ggplot(aes(x = booths)) +
  geom_jitter(aes(y = wait), width = 0.15, height = 0, alpha = 0.3) +
  geom_line(aes(y = .fitted, color = model), data = lines_v1_v2, linewidth = 1) +
  scale_color_manual(values = variant_colors) +
  labs(
    title = "Waiting time vs. booths, variants 1 and 2",
    x     = "Voting booths",
    y     = "Waiting time (minutes)",
    color = NULL
  ) +
  theme_intror()
```



```r
variant_models <- list(
  "1: none" = model_v1,
  "2: Elig" = model_v2
)

modelplot(variant_models, coef_map = c(booths = "Booths"), draw = FALSE) |>
  ggplot(aes(x = estimate, y = fct_rev(model))) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  geom_vline(xintercept = -3, color = "firebrick", linewidth = 1) +
    geom_errorbar(aes(xmin = conf.low, xmax = conf.high), width = 0.2, color =
course_primary) +
  geom_point(size = 3, color = course_primary) +
  labs(
    title = "Estimated vs. true effect of one booth",
    x     = "Minutes of waiting time",
    y     = NULL
  ) +
  theme_intror()
```



With the size of the district held constant, the slope turns negative (-1.42 minutes per booth), but it is still far from the true effect: the open path through usual turnout pulls it towards zero.

Model variant 3: usual turnout, imagined observed

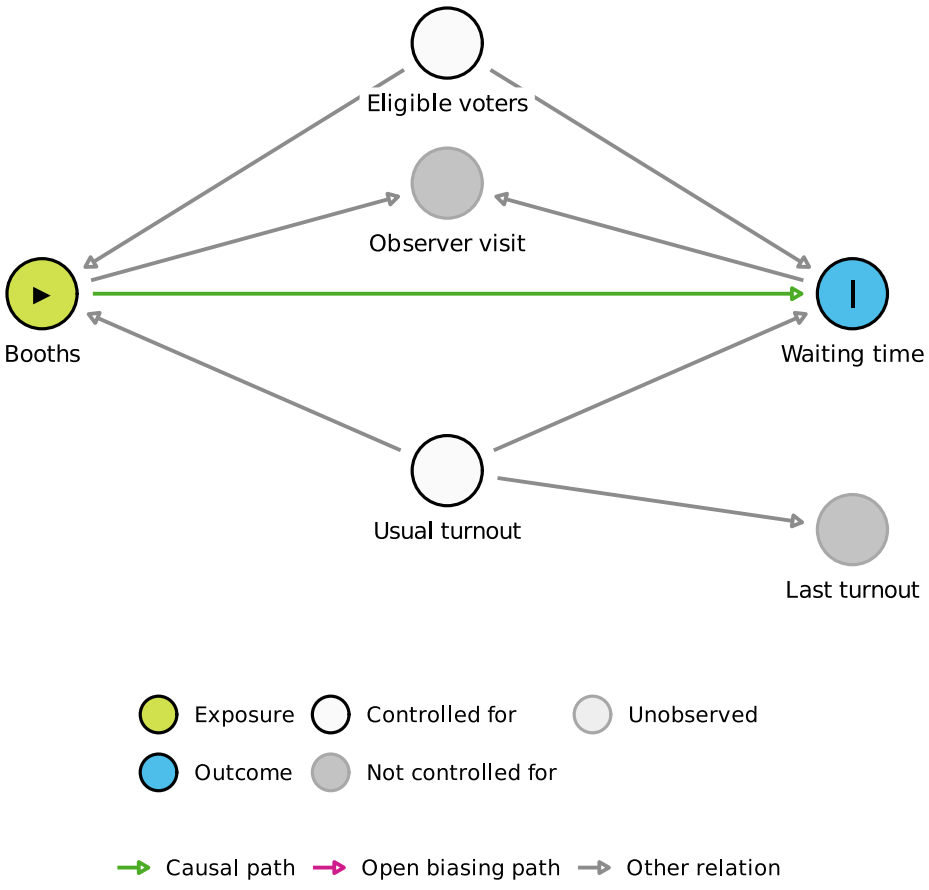
Usual turnout is the **unobserved confounder** that keeps the last backdoor path open. Imagine for one variant that the office had written down its knowledge of each district's usual turnout. Controlling for it together with eligible voters closes every backdoor path:

$$\text{Wait}_i = \beta_0 + \beta_1 \text{Booths}_i + \beta_2 \text{Eligible}_i + \beta_3 \text{UsualTurnout}_i + \varepsilon_i$$

In the DAG, usual turnout is now observed and controlled for:

```
dag_v3 <- dagitty('dag {
  booths        [exposure, pos="0,0"]
  wait          [outcome, pos="3,0"]
  eligible      [adjusted, pos="1.5,-1.7"]
  observer      [pos="1.5,-0.75"]
  usual_turnout [adjusted, pos="1.5,1.2"]
  last_turnout  [pos="3,1.6"]
  eligible      -> booths
  eligible      -> wait
  booths        -> wait
  booths        -> observer
  wait          -> observer
  usual_turnout -> wait
  usual_turnout -> last_turnout
  usual_turnout -> booths
}')
```

```
plot_dag(dag_v3, labels = node_labels)
```



```
paths(dag_v3, Z = adjustedNodes(dag_v3)) |>
  as_tibble()
```

OUTPUT

```
# A tibble: 4 x 2
  paths                            open
  <chr>                            <lgl>
1 booths -> observer <- wait       FALSE
2 booths -> wait                   TRUE
3 booths <- eligible -> wait       FALSE
4 booths <- usual_turnout -> wait FALSE
```

Only the causal path is left open. This is a valid adjustment set:

```
isAdjustmentSet(dag_v3, adjustedNodes(dag_v3))
```

OUTPUT

```
[1] TRUE
```

The office's records of usual turnout come in a separate file, which we join by district:

```
stations_usual <- read_csv("w05/usual_turnout.csv") |>
  right_join(stations, by = "district")
```

MESSAGE

```
Rows: 300 Columns: 2
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (2): district, usual_turnout

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
stations_usual
```

OUTPUT

```
# A tibble: 300 x 7
   district usual_turnout eligible last_turnout booths  wait observer
      <dbl>         <dbl>    <dbl>        <dbl>  <dbl> <dbl>    <dbl>
 1        1          70.8     1760         74.5      4    16        1
 2        2          67       1653         67.4      3    12        0
 3        3          81.5     1995         84.4      5    16        1
 4        4          59.6     1293         60.6      1    15        1
 5        5          77.2     1847         72.6      4    18        1
 6        6          73.8     1173         78.4      2     6        0
 7        7          63.4     1457         65.4      2     8        0
 8        8          69       2120         70.3      4    21        1
 9        9          67.8     1908         68        3    17        1
10       10          65.5     2211         70.6      4    16        1
# i 290 more rows
```

```
model_v3 <- lm(wait ~ booths + eligible + usual_turnout, data = stations_usual)
model_v3 |>
  tidy()
```

OUTPUT

```
# A tibble: 4 x 5
  term          estimate std.error statistic                          p.value
  <chr>            <dbl>     <dbl>     <dbl>                            <dbl>
1 (Intercept)   -19.7      3.93        -5.01                          9.47e- 7
2 booths         -2.98     0.452       -6.60                          1.91e-10
3 eligible        0.0155   0.00105     14.8                           1.66e-37
4 usual_turnout   0.225    0.0532       4.22                          3.20e- 5
```

```
beta_v3 <- model_v3 |>
  tidy() |>
  filter(term == "booths") |>
  pull(estimate)
beta_v3
```

OUTPUT

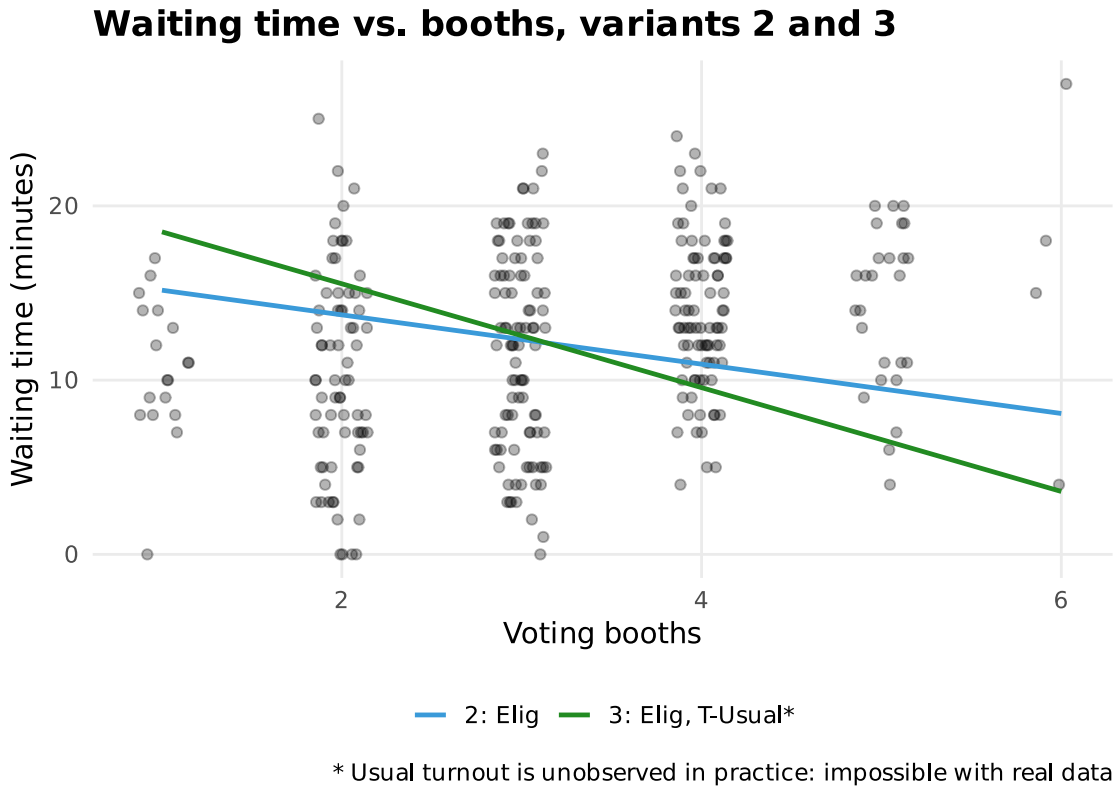
```
[1] -2.980061
```

```
usual_grid <- stations_usual |>
  summarize(usual_turnout = mean(usual_turnout)) |>
  expand_grid(prediction_grid)

lines_v3 <- model_v3 |>
  augment(newdata = usual_grid) |>
  mutate(model = "3: Elig, T-Usual*")

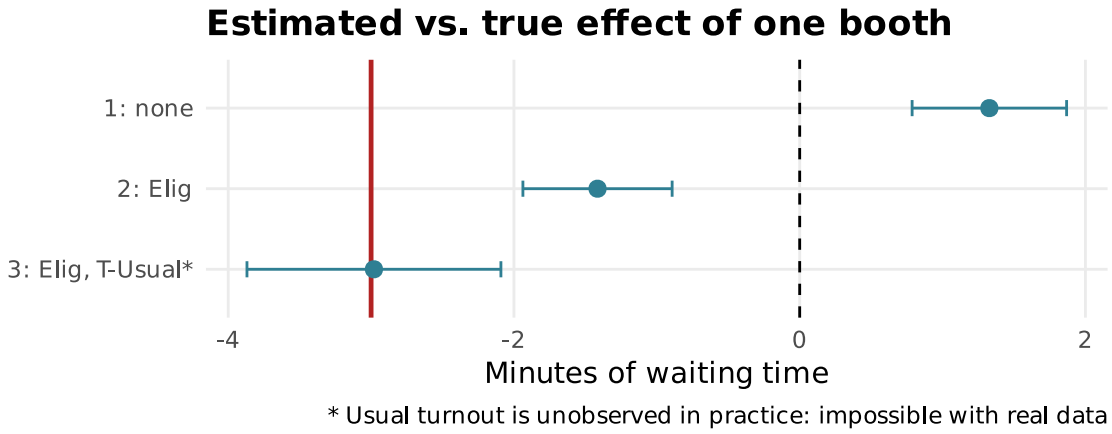
lines_v2_v3 <- bind_rows(lines_v2, lines_v3)
```

```
stations |>
  ggplot(aes(x = booths)) +
  geom_jitter(aes(y = wait), width = 0.15, height = 0, alpha = 0.3) +
  geom_line(aes(y = .fitted, color = model), data = lines_v2_v3, linewidth = 1) +
  scale_color_manual(values = variant_colors) +
  labs(
    title   = "Waiting time vs. booths, variants 2 and 3",
    x       = "Voting booths",
    y       = "Waiting time (minutes)",
    color   = NULL,
    caption = "* Usual turnout is unobserved in practice: impossible with real data"
  ) +
  theme_intror()
```



```r
variant_models <- list(
  "1: none"           = model_v1,
  "2: Elig"           = model_v2,
  "3: Elig, T-Usual*" = model_v3
)

modelplot(variant_models, coef_map = c(booths = "Booths"), draw = FALSE) |>
  ggplot(aes(x = estimate, y = fct_rev(model))) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  geom_vline(xintercept = -3, color = "firebrick", linewidth = 1) +
    geom_errorbar(aes(xmin = conf.low, xmax = conf.high), width = 0.2, color =
course_primary) +
  geom_point(size = 3, color = course_primary) +
  labs(
    title   = "Estimated vs. true effect of one booth",
    x       = "Minutes of waiting time",
    y       = NULL,
    caption = "* Usual turnout is unobserved in practice: impossible with real data"
  ) +
  theme_intror()
```



With a valid adjustment set, the estimate (-2.98 minutes per booth) lands right on the true effect. In practice, usual turnout is not recorded, so from here on it is unobserved again.

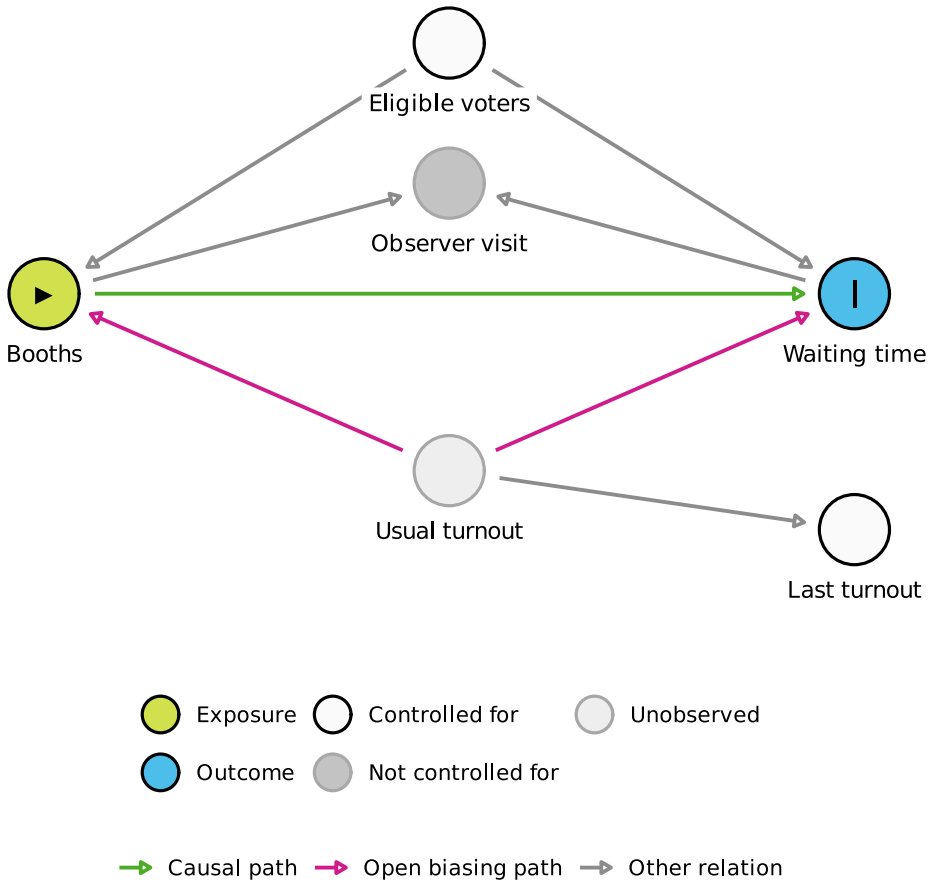
Model variant 4: last turnout as a proxy

Without usual turnout, we can still control for last election's turnout. It is a **proxy**: it is caused by usual turnout, so it carries part of the same information:

$$\text{Wait}_i = \beta_0 + \beta_1 \text{Booths}_i + \beta_2 \text{Eligible}_i + \beta_3 \text{LastTurnout}_i + \varepsilon_i$$

```
dag_v4 <- dagitty('dag {
  booths        [exposure, pos="0,0"]
  wait          [outcome, pos="3,0"]
  eligible      [adjusted, pos="1.5,-1.7"]
  observer      [pos="1.5,-0.75"]
  usual_turnout [latent, pos="1.5,1.2"]
  last_turnout  [adjusted, pos="3,1.6"]
  eligible      -> booths
  eligible      -> wait
  booths        -> wait
  booths        -> observer
  wait          -> observer
  usual_turnout -> wait
  usual_turnout -> last_turnout
  usual_turnout -> booths
}')
```

```
plot_dag(dag_v4, labels = node_labels)
```



Last turnout doesn't sit on the backdoor path, so `dagitty` still reports the path open:

```
paths(dag_v4, Z = adjustedNodes(dag_v4)) |>
  as_tibble()
```

OUTPUT

```
# A tibble: 4 x 2
  paths                            open
  <chr>                            <lgl>
1 booths -> observer <- wait       FALSE
2 booths -> wait                   TRUE
3 booths <- eligible -> wait       FALSE
4 booths <- usual_turnout -> wait TRUE
```

```
isAdjustmentSet(dag_v4, adjustedNodes(dag_v4))
```

OUTPUT

```
[1] FALSE
```

```
model_v4 <- lm(wait ~ booths + eligible + last_turnout, data = stations)
model_v4 |>
  tidy()
```

OUTPUT

```
# A tibble: 4 x 5
  term         estimate std.error statistic                                p.value
  <chr>           <dbl>     <dbl>     <dbl>                                  <dbl>
1 (Intercept)  -14.7     3.45         -4.25                                2.92e- 5
2 booths        -2.45    0.406        -6.05                                4.48e- 9
3 eligible       0.0145  0.000980     14.8                                 1.58e-37
4 last_turnout   0.154   0.0462        3.34                                9.41e- 4
```

```
beta_v4 <- model_v4 |>
  tidy() |>
  filter(term == "booths") |>
  pull(estimate)
beta_v4
```

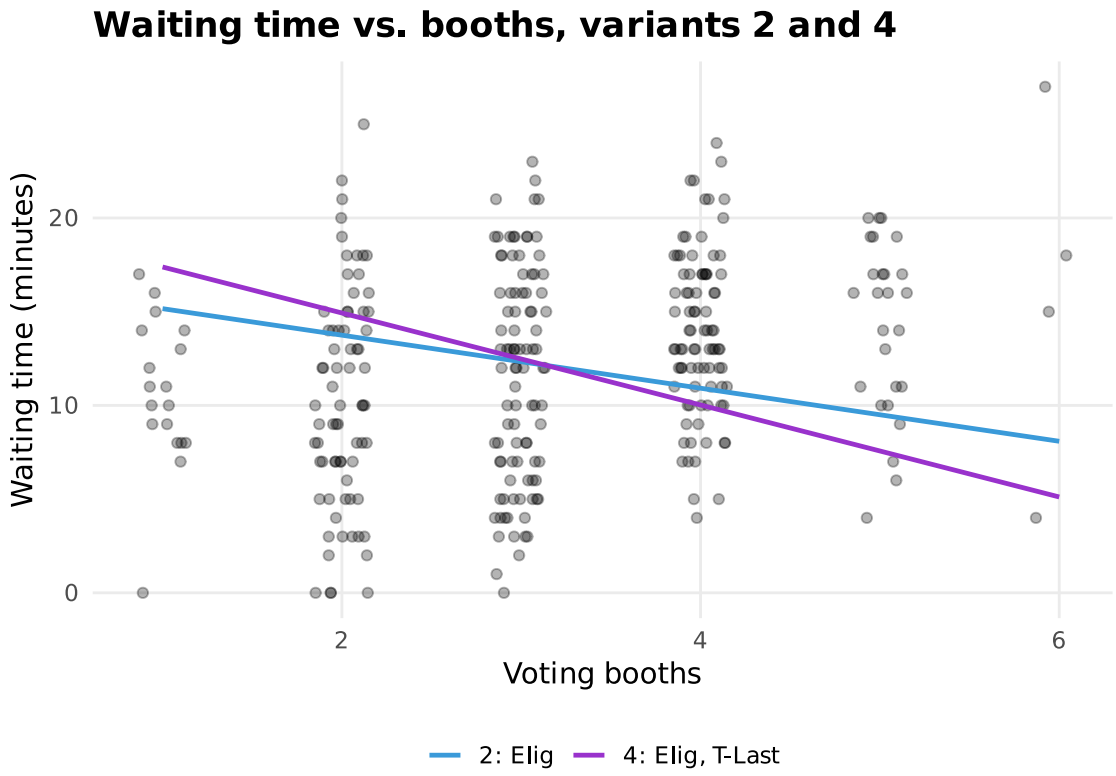
OUTPUT

```
[1] -2.454241
```

```
lines_v4 <- model_v4 |>
  augment(newdata = prediction_grid) |>
  mutate(model = "4: Elig, T-Last")

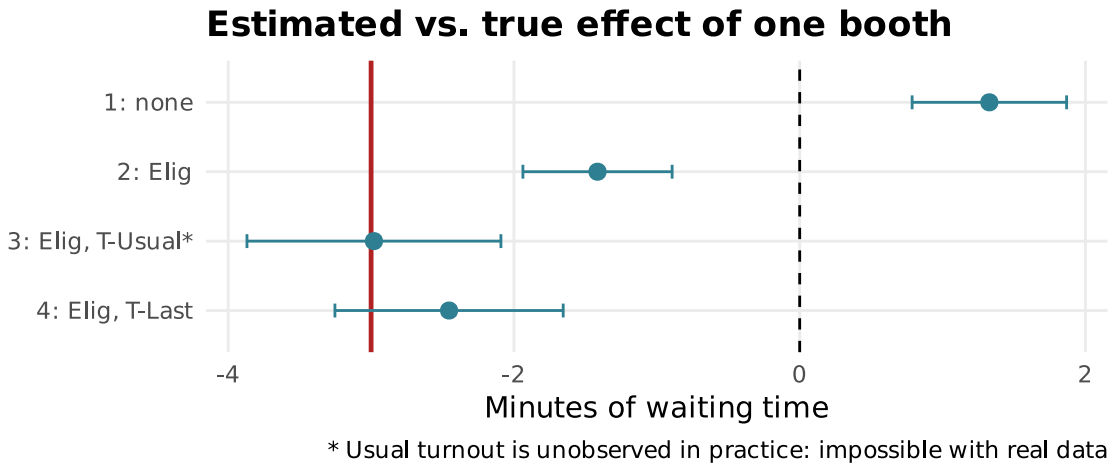
lines_v2_v4 <- bind_rows(lines_v2, lines_v4)
```

```r
stations |>
  ggplot(aes(x = booths)) +
  geom_jitter(aes(y = wait), width = 0.15, height = 0, alpha = 0.3) +
  geom_line(aes(y = .fitted, color = model), data = lines_v2_v4, linewidth = 1) +
  scale_color_manual(values = variant_colors) +
  labs(
    title = "Waiting time vs. booths, variants 2 and 4",
    x     = "Voting booths",
    y     = "Waiting time (minutes)",
    color = NULL
  ) +
  theme_intror()
```



```r
variant_models <- list(
  "1: none"           = model_v1,
  "2: Elig"           = model_v2,
  "3: Elig, T-Usual*" = model_v3,
  "4: Elig, T-Last"   = model_v4
)

modelplot(variant_models, coef_map = c(booths = "Booths"), draw = FALSE) |>
  ggplot(aes(x = estimate, y = fct_rev(model))) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  geom_vline(xintercept = -3, color = "firebrick", linewidth = 1) +
    geom_errorbar(aes(xmin = conf.low, xmax = conf.high), width = 0.2, color =
course_primary) +
  geom_point(size = 3, color = course_primary) +
  labs(
    title   = "Estimated vs. true effect of one booth",
    x       = "Minutes of waiting time",
    y       = NULL,
    caption = "* Usual turnout is unobserved in practice: impossible with real data"
  ) +
  theme_intror()
```



The proxy moves the estimate from -1.42 to -2.45 minutes per booth, much closer to the true effect. It narrows the gap the unobserved confounder leaves, but can't close it.

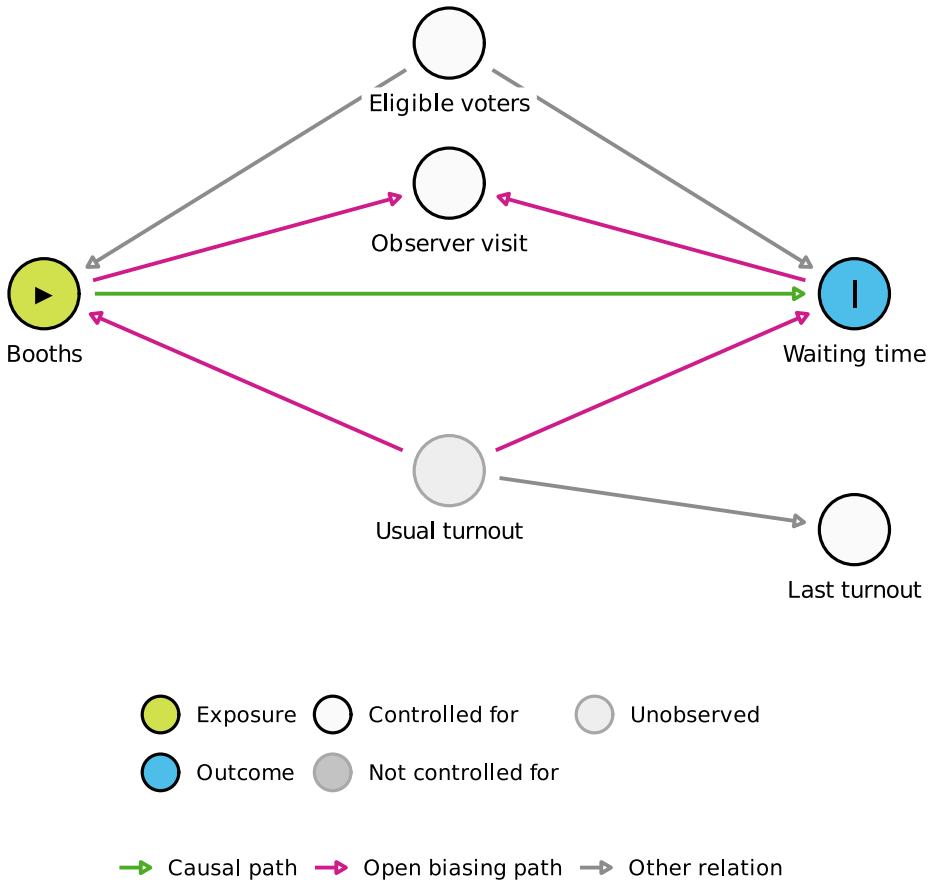
Model variant 5: adding the observer visit

The observer visit is a **collider**: both booths and waiting time cause it. Still, why not control for it on top of variant 4, as a measure of how closely the office watched a station?

$$\text{Wait}_i = \beta_0 + \beta_1 \text{Booths}_i + \beta_2 \text{Eligible}_i + \beta_3 \text{LastTurnout}_i + \beta_4 \text{Observer}_i + \varepsilon_i$$

```
dag_v5 <- dagitty('dag {
  booths        [exposure, pos="0,0"]
  wait          [outcome, pos="3,0"]
  eligible      [adjusted, pos="1.5,-1.7"]
  observer      [adjusted, pos="1.5,-0.75"]
  usual_turnout [latent, pos="1.5,1.2"]
  last_turnout  [adjusted, pos="3,1.6"]
  eligible      -> booths
  eligible      -> wait
  booths        -> wait
  booths        -> observer
  wait          -> observer
  usual_turnout -> wait
  usual_turnout -> last_turnout
  usual_turnout -> booths
}')
```

```
plot_dag(dag_v5, labels = node_labels)
```



Controlling for the observer visit opens the path Booths → Observer ← Wait:

```
paths(dag_v5, Z = adjustedNodes(dag_v5)) |>
  as_tibble()
```

OUTPUT

```
# A tibble: 4 x 2
  paths                           open
  <chr>                           <lgl>
1 booths -> observer <- wait      TRUE
2 booths -> wait                  TRUE
3 booths <- eligible -> wait      FALSE
4 booths <- usual_turnout -> wait TRUE
```

```
isAdjustmentSet(dag_v5, adjustedNodes(dag_v5))
```

OUTPUT

```
[1] FALSE
```

```
model_v5 <- lm(wait ~ booths + eligible + last_turnout + observer, data = stations)
model_v5 |>
  tidy()
```

OUTPUT

```
# A tibble: 5 x 5
  term         estimate std.error statistic                              p.value
  <chr>           <dbl>     <dbl>     <dbl>                                <dbl>
1 (Intercept)  -5.14     2.55         -2.02                              4.47e- 2
2 booths       -1.35     0.300        -4.49                              1.01e- 5
3 eligible      0.00758  0.000819      9.25                              4.57e-18
4 last_turnout  0.0835   0.0335        2.49                              1.33e- 2
5 observer      6.47     0.389        16.6                               2.79e-44
```

```
beta_v5 <- model_v5 |>
  tidy() |>
  filter(term == "booths") |>
  pull(estimate)
beta_v5
```

OUTPUT

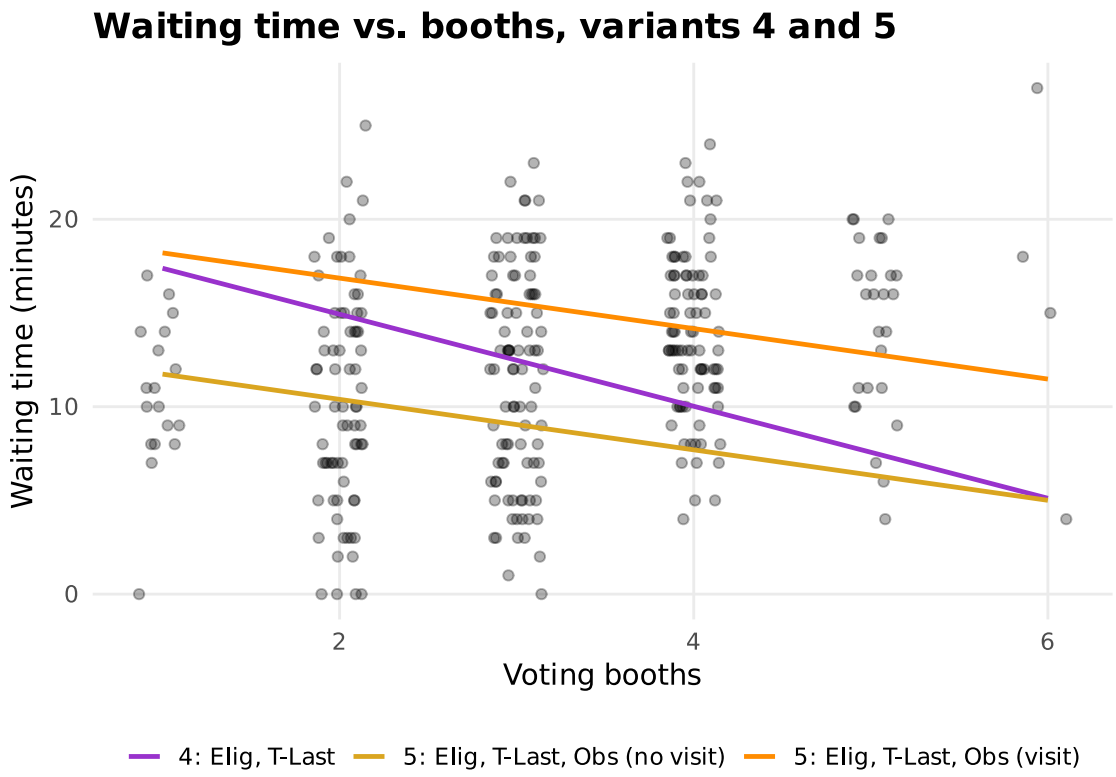
```
[1] -1.34639
```

The model predicts one line for visited and one for unvisited stations. Visited stations are mostly those with long queues, unvisited ones those with short queues, whatever their number of booths. Within each group, the booth slope flattens compared to variant 4:

```
lines_v5 <- model_v5 |>
  augment(newdata = expand_grid(prediction_grid, observer = 0:1)) |>
  mutate(model = if_else(observer == 1, "5: Elig, T-Last, Obs (visit)", "5: Elig, T-
Last, Obs (no visit)"))

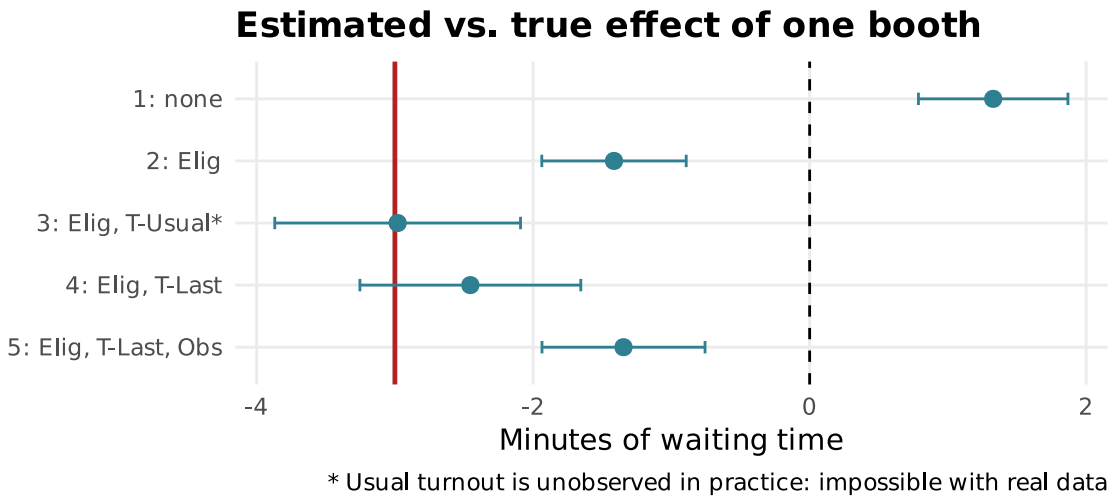
lines_v4_v5 <- bind_rows(lines_v4, lines_v5)
```

```
stations |>
  ggplot(aes(x = booths)) +
  geom_jitter(aes(y = wait), width = 0.15, height = 0, alpha = 0.3) +
  geom_line(aes(y = .fitted, color = model), data = lines_v4_v5, linewidth = 1) +
  scale_color_manual(values = variant_colors) +
  labs(
    title = "Waiting time vs. booths, variants 4 and 5",
    x     = "Voting booths",
    y     = "Waiting time (minutes)",
    color = NULL
  ) +
  theme_intror()
```



```r
variant_models <- list(
  "1: none"              = model_v1,
  "2: Elig"              = model_v2,
  "3: Elig, T-Usual*"    = model_v3,
  "4: Elig, T-Last"      = model_v4,
  "5: Elig, T-Last, Obs" = model_v5
)

modelplot(variant_models, coef_map = c(booths = "Booths"), draw = FALSE) |>
  ggplot(aes(x = estimate, y = fct_rev(model))) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  geom_vline(xintercept = -3, color = "firebrick", linewidth = 1) +
    geom_errorbar(aes(xmin = conf.low, xmax = conf.high), width = 0.2, color =
course_primary) +
  geom_point(size = 3, color = course_primary) +
  labs(
    title   = "Estimated vs. true effect of one booth",
    x       = "Minutes of waiting time",
    y       = NULL,
    caption = "* Usual turnout is unobserved in practice: impossible with real data"
  ) +
  theme_intror()
```



With the observer visit controlled for, a booth seems to save only 1.35 minutes: the office would conclude that extra booths hardly pay off. The collider undid most of what the proxy gained.

Overview

Regression table

All five model variants side by side:

```r
modelsummary(
  variant_models,
  output    = "tinytable",
  coef_map  = c(
    "booths"        = "Booths",
    "eligible"      = "Eligible voters",
    "usual_turnout" = "Usual turnout (%)",
    "last_turnout"  = "Last turnout (%)",
    "observer"      = "Observer visit"
  ),
  statistic = NULL,
  stars     = FALSE,
  gof_map   = c("nobs", "r.squared"),
  fmt       = 3,
  notes     = "* Usual turnout is unobserved in practice: variant 3 is impossible with real data."
)
```

	1: none	2: Elig	3: Elig, Usual*	4: Elig, Last	5: Elig, Obs
Booths	1.328	-1.415	-2.980	-2.454	-1.346
Eligible voters		0.012	0.016	0.015	0.008
Usual turnout (%)			0.225		
Last turnout (%)				0.154	0.083
Observer visit					6.469
Num.Obs.	300	300	300	300	300
R2	0.073	0.500	0.529	0.518	0.751

* Usual turnout is unobserved in practice: variant 3 is impossible with real data.

Note that R^2 rises with every control: a better fit says nothing about whether a control belongs in the model.

Checking adjustment sets in one run

Every variant's DAG code marks its controls `[adjusted]`. `isAdjustmentSet()` checks whether a set we chose is valid, so one `rowwise()` pass checks all five variants at once:

```r
variant_dags <- tibble(
  variant = names(variant_models),
  dag     = list(dag_v1, dag_v2, dag_v3, dag_v4, dag_v5)
)

variant_dags |>
  rowwise() |>
  mutate(
    controls = str_flatten_comma(adjustedNodes(dag)),
    valid    = isAdjustmentSet(dag, adjustedNodes(dag))
  ) |>
  ungroup() |>
  select(variant, controls, valid)
```

OUTPUT

```
# A tibble: 5 x 3
  variant              controls                           valid
  <chr>                <chr>                              <lgl>
1 1: none              ""                                 FALSE
2 2: Elig              "eligible"                         FALSE
3 3: Elig, T-Usual*    "eligible, usual_turnout"          TRUE
4 4: Elig, T-Last      "eligible, last_turnout"           FALSE
5 5: Elig, T-Last, Obs "eligible, last_turnout, observer" FALSE
```

We didn't need to try out control sets at all. `adjustmentSets()` finds the valid ones from the DAG alone: it ignores `[adjusted]` marks and only needs to know which variables are observed. For the original DAG B, with usual turnout unobserved, it finds none:

```r
dag_b |>
  adjustmentSets() |>
  length()
```

OUTPUT

```
[1] 0
```

In variant 3′s DAG, usual turnout is observed, and `adjustmentSets()` finds exactly one minimal set:

```r
adjustmentSets(dag_v3)
```

OUTPUT

```
{ eligible, usual_turnout }
```

With `type = "all"`, it lists every valid set, not just the smallest ones:

```r
adjustmentSets(dag_v3, type = "all")
```

```
>_ OUTPUT
{ eligible, usual_turnout }
{ eligible, last_turnout, usual_turnout }
```

The true effect

The polling stations aren't real: we simulated them, following DAG B. That's how we know the true effect, which real data never tells us. The code that generated the data:

```r
p_load(tidyverse)
set.seed(175)
n_districts <- 300

# eligible voters per district
eligible_mean <- 1600
eligible_sd   <- 380
eligible_min  <- 800
eligible_max  <- 2500

# usual turnout (%): the voting habit of the district, known to the office, normally never recorded
turnout_mean       <- 72
turnout_sd         <- 7
turnout_min        <- 52
turnout_max        <- 88
last_turnout_noise <- 2.5

# one booth per ~500 eligible voters, plus one per 10 points of usual turnout
voters_per_booth  <- 500
turnout_per_booth <- 10
booths_noise      <- 0.45
booths_min        <- 1
booths_max        <- 6
booths_mean       <- 3.2

# waiting time (minutes) at an average station, and what drives it
wait_mean         <- 12
wait_per_eligible <- 0.016
wait_per_turnout  <- 0.25
true_effect       <- -3
wait_noise        <- 4

# observers go to every station with the most booths and to every long queue
observer_wait <- 12

polling_stations <- tibble(
  district      = 1:n_districts,
  eligible      = rnorm(n_districts, eligible_mean, eligible_sd) |>
    pmax(eligible_min) |>
    pmin(eligible_max) |>
    round(),
  usual_turnout = rnorm(n_districts, turnout_mean, turnout_sd) |>
    pmax(turnout_min) |>
    pmin(turnout_max),
  last_turnout  = round(usual_turnout + rnorm(n_districts, 0, last_turnout_noise), 1),
  booths        = (eligible / voters_per_booth +
    (usual_turnout - turnout_mean) / turnout_per_booth +
    rnorm(n_districts, 0, booths_noise)) |>
    round() |>
    pmax(booths_min) |>
```

```
    pmin(booths_max),
  wait          = (wait_mean +
    wait_per_eligible * (eligible - eligible_mean) +
    wait_per_turnout * (usual_turnout - turnout_mean) +
    true_effect * (booths - booths_mean) +
    rnorm(n_districts, 0, wait_noise)) |>
    pmax(0) |>
    round(),
  observer      = as.integer(booths == booths_max | wait > observer_wait)
)

polling_stations |>
  select(-usual_turnout) |>
  write_csv("w05/polling_stations.csv")

polling_stations |>
  transmute(district, usual_turnout = round(usual_turnout, 1)) |>
  write_csv("w05/usual_turnout.csv")
```

Every additional booth cuts the wait by exactly 3 minutes. Each booth estimate with its 95% CI, as a share of the true effect, and whether the CI covers the true effect:

```
variant_estimates <- variant_models |>
  modelplot(
    coef_map = c(booths = "Booths"),
    draw     = FALSE
  ) |>
  rowwise() |>
  mutate(
    share_of_true  = estimate / true_effect,
    true_within_ci = between(true_effect, conf.low, conf.high)
  ) |>
  ungroup() |>
  select(model, estimate, conf.low, conf.high, share_of_true, true_within_ci)
variant_estimates
```

OUTPUT

```
# A tibble: 5 x 6
  model                estimate conf.low conf.high share_of_true true_within_ci
  <fct>                   <dbl>    <dbl>     <dbl>         <dbl> <lgl>
1 1: none                  1.33    0.787     1.87         -0.443 FALSE
2 2: Elig                 -1.42   -1.94     -0.893         0.472 FALSE
3 3: Elig, T-Usual*       -2.98   -3.87     -2.09          0.993 TRUE
4 4: Elig, T-Last         -2.45   -3.25     -1.66          0.818 TRUE
5 5: Elig, T-Last, Obs    -1.35   -1.94     -0.757         0.449 FALSE
```

Depending on what we control for, the same data tell very different stories. Without controls, the estimate even has the wrong sign: booths seem to *lengthen* the queue. Controlling for eligible voters alone, or adding the observer visit on top of the proxy, recovers only about half of the true effect. The proxy gets to about four fifths, and its CI still covers the true effect. Only the valid adjustment set of variant 3 hits the true effect, and it needed a variable we don't have.

Concluding remarks

Which variables to control for follows from a causal graph, and the graph is an assumption: here, a single disputed arrow decided whether any adjustment set exists at all. Papers make the same assumptions whenever they "control for" something, usually without drawing them. Some confounders are unobserved; a proxy can narrow the gap they leave, but not close it. Controlling for the wrong variable, like a collider, can make an estimate worse, and no fit statistic warns you. Without a full course in causal inference, writing the DAG down in `dagitty` and checking its adjustment sets is a minimum. Standard errors and significance come in Week 6.

Try it: AfD support, turnout and density

Your task: estimate the turnout slope for AfD support while holding population density constant (on a log scale, as in Week 4) instead of East. Purge `log(density)` from both variables by hand, regress the residuals on each other, and confirm the slope with `lm()`. How does it compare to the slope with `east` held constant?

```
afd_density_model <- lm(afd ~ log(density), data = counties)
turnout_density_model <- lm(turnout ~ log(density), data = counties)

partialled_density <- afd_density_model |>
  augment(data = counties) |>
  select(county_code, density, turnout, afd_resid = .resid)
partialled_density <- turnout_density_model |>
  augment(data = partialled_density) |>
  select(county_code, afd_resid, turnout_resid = .resid)

lm(afd_resid ~ turnout_resid, data = partialled_density) |>
  tidy()
```

OUTPUT

```
# A tibble: 2 x 5
  term                     estimate std.error           statistic        p.value
  <chr>                       <dbl>     <dbl>               <dbl>          <dbl>
1 (Intercept)   -0.0000000000000452    0.264   -0.000000000000171      1.000e+ 0
2 turnout_resid -1.81                  0.0916 -19.7                    6.16 e-61
```

```
lm(afd ~ turnout + log(density), data = counties) |>
  tidy()
```

OUTPUT

```
# A tibble: 3 x 5
  term         estimate std.error statistic                              p.value
  <chr>           <dbl>     <dbl>     <dbl>                                <dbl>
1 (Intercept)    200.      8.05        24.9                              6.96e-83
2 turnout         -1.81    0.0917     -19.7                              8.69e-61
3 log(density)    -5.03    0.250      -20.1                              1.38e-62
```

Try it: the model variants under DAG A

Your task: suppose DAG A is the true DAG instead. For the control sets of model variants 1, 2, 4 and 5 (the ones that don't need usual turnout), check with `isAdjustmentSet()` whether each would be a valid adjustment set under DAG A and under DAG B. Which variants would then have estimated the effect without bias?

```
control_sets <- tibble(
  variant  = c("1: none", "2: Elig", "4: Elig, T-Last", "5: Elig, T-Last, Obs"),
  controls = list(
    character(),
    "eligible",
    c("eligible", "last_turnout"),
    c("eligible", "last_turnout", "observer")
  )
)

control_sets |>
  rowwise() |>
  mutate(
    valid_a = isAdjustmentSet(dag_a, controls),
    valid_b = isAdjustmentSet(dag_b, controls)
  ) |>
  ungroup() |>
  select(variant, valid_a, valid_b)
```

OUTPUT

```
# A tibble: 4 x 3
  variant              valid_a valid_b
  <chr>                <lgl>   <lgl>
1 1: none              FALSE   FALSE
2 2: Elig              TRUE    FALSE
3 4: Elig, T-Last      TRUE    FALSE
4 5: Elig, T-Last, Obs FALSE   FALSE
```

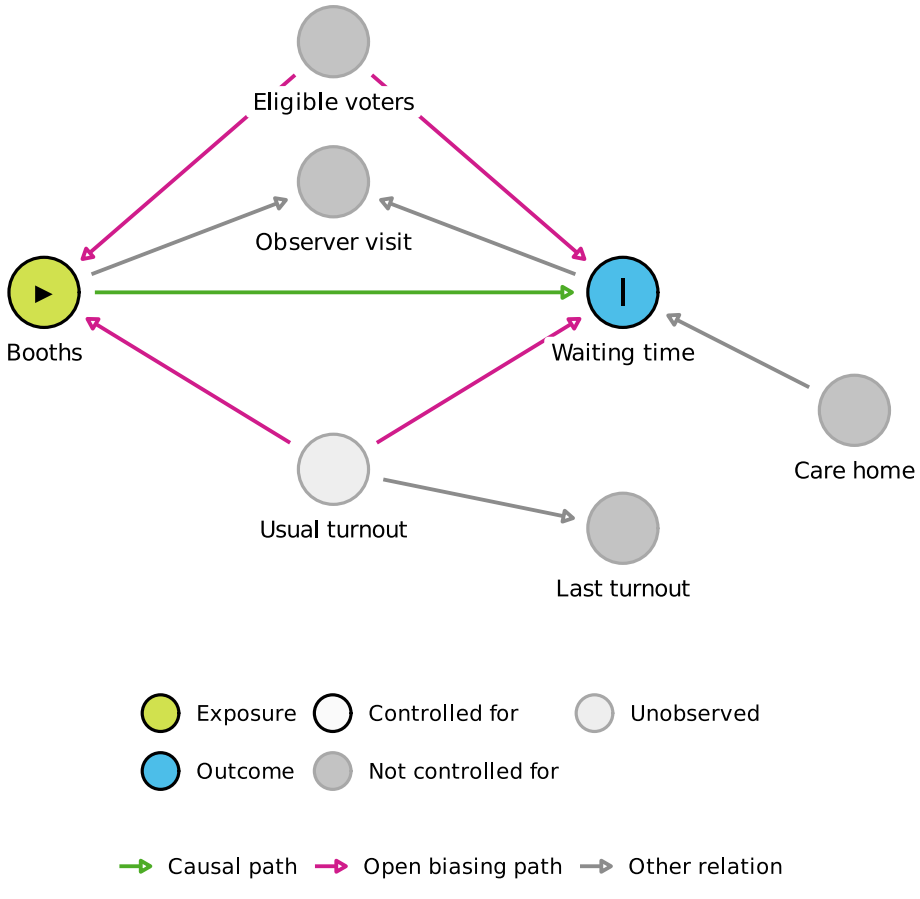
Try it: a care home in the district

Your task: some districts include a care home whose residents often need assistance in the booth, which slows down the queue. Add a node `care_home` to DAG B (with an arrow into waiting time only) and plot it. Would you need to control for it? Would controlling for it hurt? Use `paths()` and `adjustmentSets()` to argue.

```
dag_care <- dagitty('dag {
  booths        [exposure, pos="0,0"]
  wait          [outcome, pos="3,0"]
  eligible      [pos="1.5,-1.7"]
  observer      [pos="1.5,-0.75"]
  usual_turnout [latent, pos="1.5,1.2"]
  last_turnout  [pos="3,1.6"]
  care_home     [pos="4.2,0.8"]
  eligible      -> booths
  eligible      -> wait
  booths        -> wait
  booths        -> observer
  wait          -> observer
  usual_turnout -> wait
  usual_turnout -> last_turnout
  usual_turnout -> booths
  care_home     -> wait
}')

care_labels <- node_labels |>
  add_row(name = "care_home", label = "Care home")

plot_dag(dag_care, labels = care_labels)
```



```
dag_care |>
  paths() |>
  as_tibble()
```

OUTPUT

```
# A tibble: 4 x 2
  paths                            open
  <chr>                            <lgl>
1 booths -> observer <- wait       FALSE
2 booths -> wait                   TRUE
3 booths <- eligible -> wait       TRUE
4 booths <- usual_turnout -> wait TRUE
```

```
dag_care |>
  adjustmentSets() |>
  length()
```

OUTPUT

```
[1] 0
```